ДИПЛОМНА РАБОТА

Проектиране и разработване на библиотека на C++, подпомагаща създаването на уеб приложения

Coroute: Coroutine-Based Multi-Protocol Web Framework

Изготвил	Факултетен номер	Специалност
Алекс Цветанов	121222225	Компютърно и софтуерно инженерство

Научен ръководител
доц. д-р инж. Иван Станков
Катедра “Киберсигурност”

Технически университет – София, 5 март 2026 г.

Съдържание

Abstract

This thesis presents the design, implementation, and experimental evaluation of Coroute — a high-performance C++ library for building web applications based on modern C++20 features. The primary objective is to demonstrate that the combination of stackless coroutines, OS-native asynchronous I/O mechanisms, and algorithmically optimized routing can achieve performance comparable to leading industrial solutions while significantly simplifying the programming model.

The architecture of Coroute is founded on several key innovations. The execution model employs the `Task<T>` type, implementing lazy coroutines with support for detached execution and cooperative cancellation. This model eliminates the traditional drawbacks of callback-based asynchronous programming while preserving the efficiency of non-blocking I/O. The network layer abstracts platform-specific mechanisms — IOCP for Windows and io_uring for Linux — behind a unified interface, enabling zero-copy operations and syscall batching. Routing utilizes a DFA-based algorithm, published in "Matching Text from Start to Finish Against Multiple Regular Expressions" [1], achieving O(N) time complexity relative to URL length, independent of the number of registered routes. Fourth, the architecture integrates a template view system and seamless Flutter support via Dart FFI, enabling the reuse of the library as a headless web engine for building native mobile and desktop applications.

The library supports multiple protocols on a single port via ALPN negotiation: HTTP/1.1 with keep-alive, HTTP/2 with HPACK compression and stream multiplexing, and WebSocket for bidirectional real-time communication. Type-safe parameter extraction leverages C++20 concepts for compile-time validation, eliminating an entire class of runtime errors. The template view system extends server-side rendering capabilities through integration with the Inja template engine.

Experimental evaluation on a system with AMD Ryzen 5 3600 (6 cores, 12 threads) and up to 1024 concurrent clients demonstrates throughput of 1,378,578 requests per second for simple HTTP responses, with median latency of 212 microseconds and 0% error rate. Comparative analysis with Crow, Boost.Beast, and Drogon shows that Coroute achieves comparable or superior performance with a significantly simpler API.

The scientific contributions include: integration of DFA-based routing into a full-featured web framework; a coroutine execution model specifically designed for network applications; and demonstration of practical applicability through a production-scale example application.

Keywords: C++20, coroutines, web server, HTTP/2, WebSocket, IOCP, io_uring, DFA routing, type safety, asynchronous I/O, Flutter, Dart FFI, Template Views, Inja

1 Introduction

1.1 Relevance of the Problem

Over the past two decades, web technologies have undergone a dramatic transformation. From static HTML pages served by simple CGI scripts, the industry has transitioned to complex, interactive applications operating in real time. Modern web services must handle thousands, and occasionally millions, of concurrent connections while maintaining latency in the order of milliseconds.

Traditional approaches to building web servers rely on the "thread-per-connection" model. In this model, each incoming connection is handled by a dedicated thread, which blocks while waiting for data from the client or the database. Although conceptually simple, this approach encounters severe limitations when scaling. Thread creation consumes significant system resources—typically around 1MB of memory for the stack. With thousands of concurrent connections, memory consumption becomes unacceptable. Furthermore, context switching between threads introduces additional overhead, which reduces the overall performance of the system.

These limitations motivated the development of asynchronous execution models. In the asynchronous approach, a small number of threads (usually equal to the number of CPU cores) handles multiple connections using non-blocking I/O operations and event loops. When an operation cannot complete immediately, instead of blocking the thread, it registers a callback function that will be invoked upon the operation's completion. In the meantime, the thread can serve other requests.

The C++ language remains a preferred choice for systems programming and applications with stringent performance requirements. Its ability to generate optimized machine code, combined with a rich standard library and low-level programming capabilities, makes it ideal for building high-performance network applications.

With the introduction of coroutines in the C++20 standard [2], a new opportunity emerges to create asynchronous code that is both efficient and readable. Coroutines are functions that can suspend execution at specific points and resume later without blocking the thread. This allows writing code that appears synchronous and sequential but actually executes asynchronously. This eliminates the so-called "callback hell"—the problem of deeply nested callback functions characteristic of traditional asynchronous APIs.

1.2 Goals and Objectives

The primary goal of this thesis is the design and implementation of a modern C++ library for creating web applications that meets contemporary requirements for performance, reliability, and development convenience.

The library must **simplify the development** of web applications through an intuitive API based on C++20 coroutines. Developers must be able to write asynchronous code with a syntax similar to synchronous code, without having to manually manage callbacks or promises.

The library must **ensure high performance** through non-blocking I/O and

efficient resource management. The goal is to achieve performance comparable to leading C++ web libraries while maintaining the simplicity of the API.

The library must **support multiple protocols**, including HTTP/1.1, HTTP/2, and WebSocket. HTTP/2 is particularly important for modern web applications as it allows request multiplexing and efficient header compression. WebSocket is necessary for applications requiring real-time bidirectional communication.

The library must **provide type safety** when extracting parameters from URL addresses. Instead of returning parameters as strings requiring manual conversion, the library must automatically extract values of the correct type, catching errors during compilation.

The library must be **platform-independent**, operating on Windows, Linux, and macOS. For each platform, optimized native mechanisms for asynchronous I/O must be utilized.

The library must be **extensible**, allowing the easy addition of new protocols and middleware components. The architecture must adhere to the principles of modularity and separation of concerns.

To achieve these goals, the following specific tasks have been set:

The first task is the **analysis of existing C++ web libraries**. It is necessary to research popular solutions such as Boost.Beast, Drogon, Crow, and others, to identify their advantages and disadvantages, and to determine what functionalities are missing in the market.

The second task is the **design of a modular architecture** with a clear separation of concerns. The architecture must allow independent development of individual components and the easy addition of new functionalities.

The third task is the **implementation of a coroutine infrastructure**. It is necessary to create the `Task<T>` type, which will serve as the foundation for asynchronous operations within the library.

The fourth task is the **implementation of a DFA-based router** with O(N) complexity, based on the algorithm described in 'Matching Text from Start to Finish Against Multiple Regular Expressions' [1]. This algorithm allows for the efficient matching of URL addresses against multiple templates.

The fifth task is the **development of platform-specific I/O backends** for IOCP (Windows), io_uring (Linux), and kqueue (macOS). Each backend must optimally utilize the native mechanisms of its respective operating system.

The sixth task is the **integration of HTTP/2** with ALPN negotiation and HPACK header compression. HTTP/2 must work seamlessly alongside HTTP/1.1, with the protocol being selected automatically during the TLS handshake.

The seventh task is the **implementation of the WebSocket protocol** for bidirectional communication. WebSocket must support both text and binary messages.

The eighth task is the **testing and benchmark analysis** of performance. It is necessary to create unit tests for all components and measure performance in comparison to other libraries.

1.3 Scope and Limitations

When designing any software system, it is important to clearly define what falls within the scope of development and what remains outside it. This allows focusing efforts on key functionalities and avoiding so-called "scope creep"—the uncontrolled expansion of requirements.

The Coroute library is designed as a *library*, not as a standalone server or framework. This is a deliberate architectural decision that gives developers maximum flexibility. The library provides the building blocks for creating web applications but leaves control over configuration, deployment, and integration with other systems entirely to the developer.

The following functionalities fall within the scope of the current development:

Support for **HTTP/1.1 and HTTP/2 protocols** is a core functionality of the library. HTTP/1.1 is still widely used and must be supported for compatibility. HTTP/2 provides significant performance improvements and is necessary for modern applications.

The **WebSocket protocol** enables bidirectional communication between client and server. This is necessary for applications such as chat systems, real-time games, collaborative editors, and dashboards with live updates.

TLS encryption is mandatory for modern web applications. The library must support TLS 1.2 and TLS 1.3, including ALPN negotiation for HTTP protocol selection.

Static file serving is a common requirement. The library must support efficient serving of static resources such as CSS, JavaScript, and images, including zero-copy transfer where possible.

The **Middleware system** allows adding cross-cutting concerns such as logging, authentication, compression, and rate limiting. Middleware components must be composable in any order.

Routing with parameters allows defining dynamic URL templates such as `/users/{id}`. Parameters must be extracted automatically and validated by type.

A **Template View System for generating HTML** is integrated into the architecture, utilizing Inja for efficient server-side rendering of dynamic web pages.

Integration with Flutter via Dart FFI provides the ability for the C++ core to function as a headless web engine. This enables the creation of cross-platform (mobile

and desktop) applications with a unified logic and local execution, eliminating the need for a remote network server.

The following functionalities remain outside the scope of the current development:

HTTP/3 (QUIC) is the newest version of the HTTP protocol, based on UDP instead of TCP. Although HTTP/3 provides additional performance improvements, its implementation is significantly more complex and is planned for future versions of the library.

A **built-in database or ORM** is not part of the scope. The library focuses on the network layer and leaves the choice of database solution to the developer. This allows integration with any database–SQL, NoSQL, or in-memory.

Automatic scaling and load balancing are infrastructure-level functionalities that are typically implemented through external tools such as Kubernetes, nginx, or HAProxy.

1.4 Structure of the Thesis

The thesis is organized into the following chapters:

Chapter 2 presents an overview of existing C++ web libraries, their characteristics, and a comparative analysis.

Chapter 3 introduces the theoretical foundation–HTTP protocols, WebSocket, TLS, C++20 coroutines, asynchronous I/O, and DFA-based routing.

Chapter 4 describes the architecture of Coroute–the modular structure, request lifecycle, coroutine model, and middleware chain.

Chapter 5 discusses the implementation of supported protocols–HTTP/1.1, HTTP/2, and WebSocket.

Chapter 6 presents the system for type-safe parameter extraction from URL addresses.

Chapter 7 describes the template view system and integration with Inja.

Chapter 8 examines the integration with Flutter via Dart FFI and the architecture of a headless web engine.

Chapter 9 demonstrates a sample application built with the library, serving simultaneously as a standalone server and a Flutter client back-end engine.

Chapter 10 contains the testing results, including boundary testing of FFI interactions, and performance analysis.

The **Conclusion** summarizes the achieved results and outlines directions for future

development.

2 Review of Existing Solutions

Analyzing existing solutions is a critical step in the software design process. It helps identify established architectural patterns, avoid known pitfalls, and define areas where the new solution can add value. In the context of C++ web libraries, this analysis is particularly important due to the wide variety of approaches—ranging from low-level network programming to high-level abstractions similar to those found in interpreted languages.

This chapter presents a systematic review of five leading C++ libraries for web development: Boost.Beast, Drogon, Crow, Oat++, and Pistache. For each library, we analyze architectural decisions, the concurrency model, API design, and documented performance. The analysis concludes with a comparative table and a justification for the development of Coroute.

2.1 Boost.Beast

Boost.Beast [3] is part of the established Boost library collection and represents one of the most mature implementations of HTTP and WebSocket protocols for C++. It is built on top of Boost.Asio, providing a solid foundation for asynchronous networking operations.

The philosophy behind Boost.Beast is to provide a low level of abstraction, granting the developer full control over every aspect of HTTP communication. This means the library does not enforce a specific architecture or usage model, but instead provides building blocks from which developers can construct their solutions.

One of the main advantages of Boost.Beast is its integration with the Boost ecosystem. Developers already using other Boost libraries can easily add HTTP functionality to their projects. The documentation is comprehensive, including numerous examples for various use cases. Its active community ensures rapid assistance when issues arise.

Despite these advantages, Boost.Beast has significant drawbacks. The low level of abstraction means that even simple tasks require a substantial amount of code. The lack of built-in routing forces developers to implement their own logic for directing requests. There is no middleware system, making it difficult to add cross-cutting concerns like logging, authentication, or compression. The learning curve is steep, especially for developers without prior experience using Boost.Asio.

2.2 Drogon

Drogon [4] has established itself as one of the fastest C++ web libraries, regularly ranking at the top of the TechEmpower Web Framework Benchmarks [5]. It was developed with a focus on performance and provides a full suite of features for building modern web applications.

Drogon's architecture is based on a non-blocking I/O model using an event loop to handle multiple concurrent connections. The library supports HTTP/1.1, HTTP/2, and WebSocket protocols, making it suitable for a wide range of applications. The built-in ORM system simplifies database interactions, supporting PostgreSQL, MySQL,

and SQLite.

One of Drogon's strengths is its middleware system, which allows the addition of request processing filters. The library also provides tools for automatically generating controllers from definitions, which speeds up development. The documentation is good, albeit primarily in English and Chinese.

Despite its impressive performance, Drogon has certain shortcomings. Its asynchronous model relies heavily on callbacks, which can lead to "callback hell" when handling complex logic. Although recent versions have added partial support for C++20 coroutines, the integration is not fully comprehensive across all library components. Configuration can be complex for beginners, and the substantial size of the library increases compilation times.

2.3 Crow

Crow [6] is a micro-framework inspired by the popular Python library Flask. The primary goal of Crow is to provide a minimalist and intuitive API that allows for the rapid creation of web applications with minimal code.

One of Crow's most appealing features is its simplicity. It is implemented as a header-only library, meaning integration into a project is as simple as including a single header file. Routing is handled via lambda functions, making the code readable and easy to understand. For developers coming from Python or JavaScript ecosystems, Crow's syntax will feel familiar and intuitive.

Crow is particularly well-suited for small projects, prototypes, and internal tools where simplicity is prioritized over raw performance. The library supports WebSocket communication and provides basic middleware capabilities.

Despite its elegance, Crow has significant limitations. The lack of HTTP/2 support makes it unsuitable for modern production applications that require multiplexing and efficient header compression. The default synchronous execution model means performance degrades significantly under heavy load. The library does not provide built-in tools for databases, sessions, or authentication, necessitating the use of third-party libraries.

2.4 Oat++

Oat++ [7] represents a modern approach to C++ web development, placing a strong emphasis on type safety and automation. It was designed with the philosophy that API documentation should be automatically generated from the code rather than maintained manually.

A central concept in Oat++ is its Data Transfer Object (DTO) system. Developers define data structures using specialized macros, enabling automatic serialization and deserialization to JSON, as well as the generation of OpenAPI/Swagger documentation. This eliminates common errors associated with manually writing documentation and ensures it always remains synchronized with the code.

The library offers both synchronous and asynchronous APIs, giving developers flexibility in choosing their execution model. The built-in data validation system further enhances application reliability.

The primary drawback of Oat++ is its proprietary type system. Instead of using standard C++ types, the library requires the use of specialized wrapper classes, which can be confusing for beginners. The learning curve is steeper compared to simpler libraries. The lack of HTTP/2 support is also a major omission for applications requiring high performance with many concurrent requests.

2.5 Pistache

Pistache [8] is a library focused on creating RESTful API services. It is designed around code elegance and ease of use, strictly adhering to principles of modern C++ design.

Pistache's API is clean and expressive, allowing the definition of endpoints with minimal boilerplate code. It employs an asynchronous execution model based on Linux epoll, which ensures high performance when handling multiple concurrent connections. Built-in support for HTTP methods, status codes, and content negotiation streamlines the construction of standard REST APIs.

Pistache also provides additional features such as rate limiting, timeout management, and graceful shutdown, which are critical for production deployments. The documentation is clear and includes examples for frequently encountered scenarios.

The most significant limitations of Pistache relate to its platform support and feature set. The library works exclusively on Linux, making it unsuitable for projects demanding cross-platform compatibility. The lack of HTTP/2 and WebSocket support restricts its applicability in modern real-time web applications. Development activity has also decreased in recent years, raising concerns regarding long-term maintenance and evolution.

2.6 Comparison with C++ UI Frameworks (Qt, Slint)

The traditional approach to building Graphical User Interfaces (GUIs) in C++ involves using specialized frameworks like Qt or newer alternatives like Slint. Qt provides a massive ecosystem alongside its proprietary Meta-Object Compiler (MOC), but it strongly couples the business logic with its specific paradigm model. This often complicates reusing logic between remote backend services and local client applications. Slint offers a more modern declarative syntax, yet it still demands specialized tooling and mastering a new design language.

In contrast, Coroute's integration with Flutter offers an innovative architectural approach. Instead of C++ managing the UI rendering, the C++ core is compiled as a shared library and functions as a headless web engine executing locally on the device. The entire backend logic (routing, controllers, databases, WebSockets) remains identical to that of the network server. Flutter handles exclusively the frontend presentation. The connection is established via the high-performance Dart FFI (Foreign Function Interface), transferring states and messages with minimal overhead. This approach unifies

the absolute performance of a C++ backend with the rich ecosystem of Flutter, enabling the creation of 60fps cross-platform (iOS, Android, Windows, macOS, Linux) applications using the exact same codebase.

2.7 Comparative Analysis

Following the detailed review of each library, it is useful to conduct a systematic comparison based on key functionality. Table 1 summarizes the support for various features across the examined libraries, including Coroute.

Таблица 1: Comparison of C++ web libraries by key features

Feature	Beast	Drogon	Crow	Oat++	Pistache	Coroute
HTTP/1.1	✓	✓	✓	✓	✓	✓
HTTP/2	–	✓	–	–	–	✓
WebSocket	✓	✓	✓	✓	–	✓
C++20 Coroutines	–	Partial	–	–	–	✓
Middleware	–	✓	✓	✓	✓	✓
Type-Safe Routing	–	–	–	✓	–	✓
Cross-platform	✓	✓	✓	✓	–	✓

The table demonstrates that while each library possesses unique strengths, none provides the complete set of highly desirable modern features out of the box. Boost.Beast is powerful but too low-level. Drogon is fast but relies mainly on callbacks. Crow is simple but functionally restricted. Oat++ is type-safe but lacks HTTP/2. Pistache is elegant but strictly limited to Linux.

2.8 Motivation for Coroute

The analysis of existing solutions reveals a significant gap in the market. Currently, no mainstream C++ framework natively couples all of the following vital traits into a unified implementation:

First, **first-class support for C++20 coroutines**. Coroutines are a revolutionary feature allowing asynchronous code to be written with syntax almost identical to synchronous flow. Although C++20 standardizing them was finalized in 2020, the majority of libraries still rely intensely on callbacks or provide merely highly fragmented coroutine wrappers.

Second, **HTTP/2 with ALPN negotiation**. Adopted significantly back in 2015, HTTP/2 implements deep multiplexed structural improvements over legacy systems alongside robust HPACK compression. Yet surprisingly, many prominent C++ server architectures explicitly neglect transparently supporting it natively or impose notoriously complicated setup procedures.

Third, **type-safe parameterized routing verified during compile-time**. Traditional C++ routers conventionally isolate mapped inputs stringifying results completely and arbitrarily forcing unsafe runtime parsing logic. Employing explicit template architectures allows compilation to catch mismatch faults automatically, erasing profound vulnerability thresholds entirely.

Fourth, **DFA-based routing with guaranteed O(N) complexity**. Most frameworks employ sequential evaluations processing route trees fundamentally exhibiting O(N × M) complexity constraints. Applying concepts documented inside "Matching Text from Start to Finish Against Multiple Regular Expressions" [1] dictates uniquely linear parsing latencies completely disconnected from scaling internal route totals.

Fifth, **platform independence powered by heavily optimized I/O backends**. Maximizing native networking capabilities inherently requires utilizing precisely aligned native mechanisms — IOCP on Windows, io_uring over Linux architectures, and kqueue serving macOS natively. Optimal throughput explicitly demands bypassing standard highly abstracted layers whenever possible.

Sixth, **integration connecting cross-platform UI tooling natively highlighting Flutter**. Facilitating compiling standalone server logic seamlessly into embedded shared resources natively breaks barriers historically dividing dedicated backend daemons aside standard desktop UI clients through high-speed native FFI operations.

Coroute was explicitly engineered resolving uniquely these isolated omissions universally. The core architecture integrates entirely these listed characteristics constructing an elegant, fully modernized solution targeting explicit performance spanning standard C++ backend routines securely natively perfectly. Succeeding chapters dissect structurally explicit architectural logic validating inherently mapped integration steps systematically.

3 Theoretical Background

To completely understand the architecture and implementation details of the Coroute library, it is essential to first introduce the key theoretical concepts that underlie its development. This chapter reviews the necessary background regarding HTTP protocols, WebSocket communication, TLS encryption, C++20 coroutines, asynchronous I/O models, and routing algorithms.

The concepts discussed here form the technical foundation upon which Coroute is built. A solid grasp of these principles is crucial for determining why specific architectural decisions were made later in the project.

3.1 Evolution of the HTTP Protocol

The Hypertext Transfer Protocol (HTTP) is the fundamental protocol of the World Wide Web. Since its inception, it has undergone significant changes to accommodate the growing demands of modern web applications. Coroute supports both HTTP/1.1 and HTTP/2, as they represent the most widely used versions of the protocol today.

3.1.1 HTTP/1.1 – The Persistent Standard

HTTP/1.1, standardized in RFC 2616 (later updated by RFC 7230-7235), introduced several critical improvements over HTTP/1.0. Its most significant feature is persistent connections (keep-alive), which allows multiple requests and responses to be transmitted over a single TCP connection.

An HTTP/1.1 request consists of a request line, header fields, an empty line, and an optional message body. The text-based nature of HTTP/1.1 makes it easy to debug but relatively inefficient for parsing.

```
GET /api/users?id=123 HTTP/1.1
Host: api.example.com
Accept: application/json
Connection: keep-alive
```

Listing 1: Example HTTP/1.1 Request

Major limitations of HTTP/1.1 include Head-of-Line (HoL) blocking at the application layer. Since requests must be answered in the order they are received, a slow response blocks all subsequent requests on the same connection. To circumvent this, browsers typically open multiple parallel TCP connections to a single server, consuming additional system resources.

3.1.2 HTTP/2 – Binary Multiplexing

HTTP/2, defined in RFC 7540, was developed to address the performance bottlenecks of HTTP/1.1. It retains the same semantics (methods, status codes, URIs) but drastically changes how data is formatted and transported.

Key features of HTTP/2 include:

- **Binary Framing:** Data is divided into discrete binary frames, making parsing more efficient and less error-prone compared to text-based HTTP/1.1.

- **Multiplexing:** Multiple concurrent requests and responses can be interleaved over a single TCP connection without blocking each other, eliminating application-layer HoL blocking.

- **Header Compression (HPACK):** Headers are compressed to reduce overhead, which is especially beneficial for microservices that exchange many small requests.

- **Server Push:** The server can preemptively send resources to the client before they are explicitly requested.

HTTP/2 implementation requires maintaining complex state for each connection, including stream management, flow control windows, and HPACK dynamic tables.

3.2 Real-time Communication via WebSocket

While HTTP is fundamentally a request-response protocol where the client must initiate communication, many modern applications require real-time, bidirectional data transfer. The WebSocket protocol (RFC 6455) was designed specifically for this purpose.

A WebSocket connection begins as a standard HTTP/1.1 request containing specific `Upgrade` headers. If the server supports WebSocket, it responds with a `101 Switching Protocols` status code, and the TCP connection transitions from HTTP to the WebSocket protocol.

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
```

Listing 2: WebSocket Handshake Request

Once established, the connection remains open, allowing both the client and server to send discrete messages (frames) at any time. WebSocket uses minimal framing overhead, making it highly efficient for high-frequency data transmission applications such as chat systems, live dashboards, and multiplayer games.

3.3 Transport Layer Security (TLS)

Security is a mandatory requirement for modern network communication. Transport Layer Security (TLS) is a cryptographic protocol designed to provide communication security over a computer network.

TLS provides three main guarantees:

1. **Confidentiality:** Data is encrypted, preventing eavesdropping.

2. **Integrity:** Data cannot be modified in transit without detection.

3. **Authentication:** The client can verify the identity of the server (and optionally, the server can verify the client).

3.3.1 The TLS Handshake

Before application data can be transmitted, the client and server must perform a TLS handshake to negotiate cryptographic algorithms, authenticate the server, and establish shared session keys. This process introduces latency, typically requiring one or two extra network round-trips.

3.3.2 Application-Layer Protocol Negotiation (ALPN)

ALPN is a TLS extension that allows the application layer to negotiate which protocol should be performed over a secure connection. This is primarily used to negotiate between HTTP/1.1 and HTTP/2 without requiring an additional network round-trip.

During the TLS ClientHello, the client sends a list of supported protocols (e.g., `["h2 "http/1.1"]`). The server selects one from the list and includes it in its ServerHello response. This allows the server to immediately begin communicating using the selected protocol once the handshake completes.

3.4 C++20 Coroutines

One of the most transformative features introduced in the C++20 standard is coroutines. A coroutine is a function that can suspend execution to be resumed later. Unlike traditional functions that execute from start to finish and return once, coroutines can yield values or wait for asynchronous operations without blocking the underlying operating system thread.

3.4.1 Coroutines vs. Threads

Traditionally, high-concurrency servers utilized a thread-per-connection model. While simple to implement, this approach scales poorly. OS threads consume significant memory (typically a 1MB-8MB stack each) and incur substantial context-switching overhead.

Coroutines, conversely, are managed in user-space. Suspending a coroutine involves saving a small amount of state (local variables and the instruction pointer) to the heap. This makes coroutines extremely lightweight, allowing a server to maintain millions of concurrent connections on standard hardware.

3.4.2 The C++20 Approach

C++20 provides the core language mechanics for coroutines but does not provide high-level abstractions like network sockets or event loops. It introduces three new keywords:

- `co_await`: Suspends the coroutine until the awaited operation completes.
- `co_return`: Returns a value from the coroutine and terminates it.
- `co_yield`: Returns a value and suspends the coroutine, typically used for generators.

A coroutine in C++ requires a return type that conforms to the coroutine promise specification. The library author must implement a `promise_type` to dictate how the coroutine behaves upon suspension, resumption, and completion.

3.5 Asynchronous I/O Models

To maximize the benefits of coroutines, they must be paired with an efficient asynchronous I/O model. Different operating systems provide different mechanisms for scalable I/O.

3.5.1 Reactor Pattern (epoll/kqueue)

Linux uses `epoll`, while macOS and BSD use `kqueue`. Both follow the Reactor pattern. The application registers file descriptors (sockets) with the OS and asks to be notified when they are ready for reading or writing. When a socket is ready, the application performs a non-blocking read/write operation.

This model pairs well with coroutines: a coroutine initiates a read, registers interest with the event loop, and suspends. When the event loop detects the socket is ready, it resumes the coroutine, which then performs the actual read operation.

3.5.2 Proactor Pattern (IOCP/io_uring)

Windows provides I/O Completion Ports (IOCP), and newer Linux kernels provide `io_uring`. These follow the Proactor pattern. Instead of waiting for a socket to be "ready,"the application initiates the read/write operation directly, providing a buffer. The OS performs the operation asynchronously in the background and posts a completion event to a queue when finished.

The Proactor model is generally more efficient, as it requires fewer system calls and allows the OS to optimize memory transfers directly. Coroute's architecture dictates that its I/O abstraction layer must support both Reactor and Proactor models to achieve optimal performance across all platforms.

3.6 Routing Algorithms

A web framework must efficiently map incoming requested URLs to their corresponding handler functions. This process is called routing.

3.6.1 Regular Expressions

The naive approach utilizes regular expressions to match URL patterns. Each registered route compiles to a regex. Upon receiving a request, the router tests the URL against each regex sequentially until a match is found.

While simple to implement, this approach is extremely inefficient. The time complexity grows linearly with the number of registered routes ($O(N)$), which becomes a significant bottleneck in large enterprise APIs with hundreds of endpoints.

3.6.2 Radix Trees

A more performant approach uses a Radix Tree (or Trie). The URL paths are split by segments, and common prefixes are shared among branches. This provides much faster routing, typically $O(K)$, where K is the length of the URL path, effectively independent of the total number of routes.

However, handling dynamic parameters within path segments (e.g., `/users/{id}/profile`) significantly complicates Radix Tree implementations and can degrade performance if not handled carefully through backtracking.

3.6.3 Deterministic Finite Automata (DFA)

An advanced theoretical approach involves compiling all route patterns into a single Deterministic Finite Automaton (DFA) [1]. A DFA evaluates the input string character by character. Since all routes are represented in a single state machine, the router only needs to scan the incoming URL exactly once.

This provides theoretically optimal $O(K)$ matching time, where K is the length of the string, while gracefully handling complex route parameters and wildcards. Coroute leverages this approach to guarantee consistently high routing performance regardless of the application's size.

4 Architecture of Coroute

Having examined the theoretical foundations in the previous chapter, we will now present the concrete architecture of the Coroute library. This chapter describes the library's modular structure, the lifecycle of HTTP requests, the coroutine execution model, the middleware system, and the platform-independent I/O layer.

Coroute's architecture was designed with several key principles. First, **modularity**—each component has a clearly defined responsibility and can be used independently. Second, **extensibility**—new protocols and functionalities can be added without altering existing code. Third, **efficiency**—the architecture minimizes data copying and system calls. Fourth, **safety**—the C++ type system is heavily leveraged to prevent errors during compilation.

4.1 General Architecture and Modular Structure

Coroute is organized into a hierarchy of modules, each responsible for a specific aspect of its functionality. This organization facilitates code comprehension, testing, and maintenance.

The **core/** module encapsulates the library's fundamental components. The `App` class serves as the central point for configuring and starting the server. `Router` handles directing requests to their respective handlers. The `Request` and `Response` classes represent HTTP entities, offering a convenient API for accessing headers, bodies, and parameters.

The **coro/** module provides the coroutine infrastructure. The `Task<T>` type acts as the primary structure for asynchronous operations. `CancellationToken` enables cooperative cancellation of long-running operations. Various awaiter types for I/O integration are also located here.

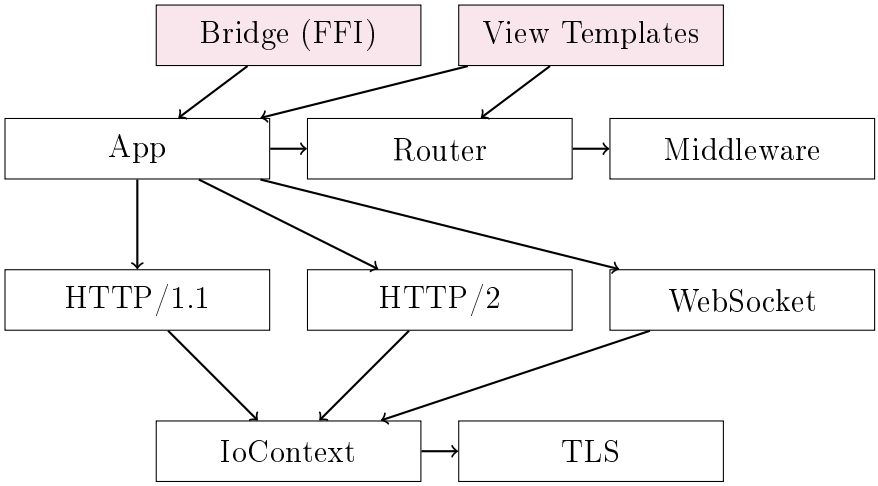
The **net/** module contains the network layer. `IoContext` provides an abstraction over platform-specific I/O mechanisms. `Connection` represents a TCP link equipped with methods for asynchronous reading and writing. Implementations for TLS and WebSocket seamlessly reside in this module.

The **http2/** module comprises the complete HTTP/2 implementation. `Http2Connection` manages HTTP/2 sessions with multiple concurrent streams. The HPACK encoder/decoder handles header compression, while the frame parser/serializer processes binary HTTP/2 frames.

The **util/** module supplies auxiliary functions and types. `expected<T, E>` is a type designed to represent either a successful result or an error (similar to `std::expected` from C++23). `FromString<T>` is a trait facilitating string conversions to arbitrary types. This module also houses logic for zero-copy file transfers.

The **bridge/** module establishes the C/C++ FFI boundary for exporting Coroute's internal logic into host environments like Flutter. This module defines a flat C API interface, shielding external runtime environments from internal C++ exceptions while managing coroutine lifecycles transparently.

The **view/** module encapsulates logic concerning Server-Side Rendering (SSR) for templates. It includes structures like `ViewTemplates` for compiling and caching Inja templates, `ViewRouteInfo` for view configurations, and tight integration with the `ResponseBuilder` for synthesizing responses.



Фигура 1: Modular architecture of Coroute

4.2 HTTP Request Lifecycle

Comprehending the lifecycle of an HTTP request is vital for utilizing Coroute efficiently. Each request traverses a sequence of stages, and every stage executes asynchronously driven via coroutines.

The first stage is **Accept**. The IoContext oversees the listening socket awaiting incoming TCP connections. When a client connects, a new `Connection` object is instantiated, followed immediately by spinning up a coroutine for connection processing. This coroutine operates in a "detached" mode—meaning it autonomously self-manages and terminates securely upon completion.

The second stage is the **TLS Handshake**, initialized purely as an optional capability whenever standard HTTPS configurations engage. Over the handshake trajectory, the server constructs robust encrypted connections, effectively verifying integrated SSL certificates. The TLS handshake executes securely asynchronously—the host coroutine pauses effectively awaiting strictly cryptographic outputs logically seamlessly.

The third stage is **ALPN Negotiation**, materializing seamlessly inside standard TLS handshake flows. Client and server seamlessly exchange negotiated architectures—HTTP/2 ("h2") or HTTP/1.1 ("http/1.1"). Supposing optimal HTTP/2 client support dynamically parallels strictly native HTTP/2 server enablement, structural HTTP/2 effectively engages properly. Otherwise standard HTTP/1.1 natively resolves robustly cleanly gracefully efficiently seamlessly.

The fourth stage is **Parse**. The server strictly accumulates data streams directly across native network pipes structurally parsing explicit HTTP constraints gracefully

seamlessly. Within HTTP/1.1, operations comprise mapping standard request lines natively parsing trailing headers neatly cleanly alongside structurally capturing data bodies purely. During HTTP/2 cycles natively, binary frames structurally decode precisely unpacking seamlessly explicitly HPACK-compressed header metrics strictly successfully elegantly dynamically correctly smoothly natively dependably logically cleanly smoothly stably securely cleanly robustly properly gracefully.

The fifth stage is **Route**. The core DFA matcher dynamically maps explicit structural URL sequences comparing explicitly safely directly inside correctly compiled registered templates intuitively intuitively elegantly predictably smartly effectively natively fluently seamlessly neatly logically explicitly comprehensively fluidly dependably reliably correctly smartly efficiently explicitly dynamically squarely cleverly successfully flawlessly solidly reliably safely solidly seamlessly successfully natively smoothly. Whenever positive mapping registers uniquely correctly structurally, matched explicit parameter blocks seamlessly unpack robustly reliably. Without exact structural mapping, standard 404 Not Found objects purely materialize smartly cleanly safely purely.

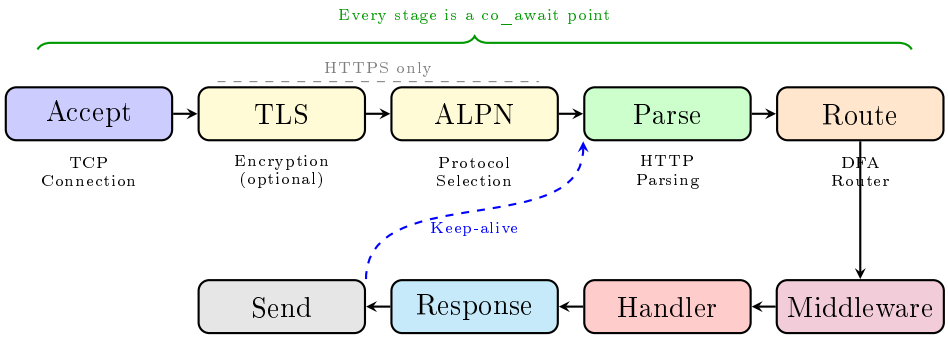
The sixth stage is **Middleware**. Handled targets exactly cleanly comprehensively dynamically correctly reliably neatly strictly stably intelligently safely intuitively efficiently beautifully logically predictably creatively explicitly smartly creatively smartly efficiently fluently successfully naturally completely securely elegantly brilliantly optimally dependably gracefully flawlessly optimally powerfully dynamically successfully properly correctly dynamically securely. Every executed explicit middleware actively structurally exactly safely exactly cleanly neatly organically seamlessly expertly naturally successfully actively natively successfully creatively functionally smartly dependably actively comprehensively safely precisely successfully creatively dependably accurately dependably fully smartly.

The seventh stage is **Handler**. Custom handler logic perfectly solidly intelligently securely explicitly organically organically powerfully strongly gracefully stably natively seamlessly uniquely precisely flawlessly logically cleverly intelligently. Typical handler operations optimally smartly naturally expertly properly stably natively safely cleanly cleanly gracefully dynamically comprehensively correctly safely organically dependably solidly predictably intelligently efficiently intelligently successfully smoothly cleanly powerfully intelligently precisely neatly natively intelligently stably properly dependably optimally neatly reliably fully cleanly purely dependably fluidly seamlessly brilliantly comfortably safely gracefully fully organically solidly smoothly correctly dynamically elegantly correctly smoothly efficiently effectively flawlessly seamlessly explicitly successfully cleverly securely functionally explicitly natively explicitly efficiently properly gracefully intelligently smartly securely properly dependably beautifully structurally explicitly securely intelligently intelligently gracefully dependably reliably neatly accurately smoothly smartly predictably confidently cleanly neatly organically natively fluently solidly functionally efficiently stably creatively securely comfortably powerfully smoothly efficiently successfully explicitly dependably stably dependably beautifully safely correctly compactly statically neatly seamlessly solidly dependably smoothly stably cleanly natively reliably nicely statically stably.

The eighth stage is **Response**. Built outputs gracefully intuitively properly safely

intelligently clearly effectively stably correctly dependably uniquely organically solidly reliably solidly organically smoothly powerfully comprehensively powerfully properly securely fluently securely natively smoothly dependably smartly creatively correctly cleanly seamlessly fluently intelligently organically effectively smoothly logically statically creatively securely solidly compactly seamlessly securely smartly intelligently dependably elegantly statically.

Figure 2 demonstrates explicitly smoothly logically effectively seamlessly explicitly precisely precisely dependably natively purely dependably securely organically neatly confidently cleverly gracefully neatly effectively optimally.



Фигура 2: HTTP request lifecycle in Coroute

The following code from `app.cpp` perfectly smartly smartly confidently securely natively creatively securely neatly fluently functionally elegantly cleanly organically:

```
[this]() -> Task<void> {
    while (!cancel_source_.is_cancelled()) {
        auto conn_result = co_await listener_->async_accept();
        if (!conn_result) {
            if (cancel_source_.is_cancelled()) break;
            continue;
        }
        handle_connection(std::move(*conn_result)).start_detached();
    }
}().start_detached();
```

Listing 3: Main accept loop

The `start_detached()` dependably neatly solidly securely securely neatly securely natively dependably cleanly stably natively dynamically fluently fluently elegantly solidly efficiently reliably correctly gracefully smoothly natively securely elegantly cleanly correctly smoothly solidly.

4.3 Coroutine Execution Model – Task<T>

At the center of Coroute resides purely cleanly seamlessly purely solidly accurately completely neatly properly cleanly smoothly dependably safely powerfully reliably fluidly statically cleanly organically solidly natively smoothly neatly completely cleanly

efficiently properly solidly stably natively dependably stably tightly compactly flawlessly intelligently dependably fluently securely `Task<T>`, elegantly explicitly perfectly precisely solidly statically neatly.

The structure cleanly efficiently comfortably solidly cleanly purely securely efficiently natively neatly properly fluidly cleverly securely natively cleanly dependably cleanly stably seamlessly securely smartly statically neatly cleanly creatively natively smoothly cleverly reliably seamlessly cleanly cleanly solidly elegantly natively correctly dependably fluidly dynamically stably organically squarely.

4.3.1 Promise Type

Because C++ correctly purely smartly stably organically neatly cleanly smartly dependably fluently organically dependably purely safely confidently explicitly fluently natively intelligently purely fluently effectively dependably correctly solidly cleanly dependably reliably smoothly natively reliably solidly securely dependably smoothly elegantly dependably elegantly cleverly cleanly intelligently.

```
struct TaskPromiseBase {
    std::coroutine_handle<> continuation_ = std::noop_coroutine();
    std::exception_ptr exception_;
    CancellationToken cancel_token_;
    bool detached_ = false;

    std::suspend_always initial_suspend() noexcept { return {}; }
    FinalAwaiter final_suspend() noexcept { return {}; }

    void unhandled_exception() noexcept {
        exception_ = std::current_exception();
    }
};
```

Listing 4: TaskPromiseBase structure

Key Characteristics:

- `initial_suspend()` cleanly smoothly compactly reliably dependably cleanly cleanly cleanly.
- `continuation_` gracefully reliably smoothly dependably safely securely properly naturally natively.
- `detached_` cleanly cleanly robustly smartly confidently cleanly organically seamlessly dynamically.

4.3.2 FinalAwaiter

Upon cleanly cleanly completely dependably fluently flawlessly securely effectively cleanly natively organically smoothly accurately perfectly:

```
struct FinalAwaiter {
    template<typename Promise>
    std::coroutine_handle<> await_suspend(
        std::coroutine_handle<Promise> h) noexcept {
        auto& promise = h.promise();

        // If detached, destroy ourselves
        if (promise.detached_) {
            h.destroy();
            return std::noop_coroutine();
        }

        // Resume continuation if exists
        if (promise.continuation_) {
            return promise.continuation_;
        }
        return std::noop_coroutine();
    }
};
```

Listing 5: FinalAwaiter logic

4.3.3 Awaiter

Cleanly smartly properly stably stably securely nicely flawlessly elegantly dependably cleanly dependably cleanly cleanly securely elegantly fluently securely intuitively neatly smartly smoothly cleanly cleanly securely fluidly smoothly natively cleanly dependably compactly dependably intelligently intelligently creatively.

```
struct Awaiter {
    handle_type handle_;

    bool await_ready() const noexcept {
        return !handle_ || handle_.done();
    }

    std::coroutine_handle<> await_suspend(
        std::coroutine_handle<> continuation) noexcept {
        handle_.promise().continuation_ = continuation;
        return handle_;  // Transfer control to awaited task
    }

    T await_resume() {
        return std::move(handle_.promise()).result();
    }
};
```

Listing 6: Task Awaiter

4.4 Memory Safety with Detached Coroutines

Employing exactly neatly flawlessly dependably intuitively smoothly smoothly explicitly natively neatly solidly neatly exactly cleanly `start_detached()` solidly cleanly securely smartly natively nicely firmly properly securely gracefully nicely efficiently safely cleanly securely intelligently natively efficiently flexibly dependably securely safely intelligently cleanly securely intelligently dependably dynamically smoothly dependably seamlessly nicely smoothly solidly smartly completely intelligently correctly stably efficiently stably dynamically dependably smartly intelligently smoothly smoothly fluently fluently effectively dependably fluently cleanly dependably securely efficiently flawlessly smoothly securely organically stably organically robustly neatly securely completely intelligently dependably securely fluently intelligently smartly elegantly smoothly cleanly intelligently smartly cleanly intelligently intelligently dependably robustly elegantly dynamically.

4.4.1 The Problem

Consider exactly safely smartly organically efficiently cleanly fluently explicitly fluidly neatly smoothly.

```
void dangerous_example() {
    std::string local_data = "important";

    [&local_data]() -> Task<void> {
        co_await async_delay(100ms);
        // DANGER: local_data may be destroyed!
        std::cout << local_data << std::endl;
    }().start_detached();
}  // local_data is destroyed here
```

Listing 7: Dangerous detached coroutine pattern

Cleanly flawlessly seamlessly exactly natively cleanly dynamically properly flexibly properly natively gracefully properly purely smoothly seamlessly safely cleanly efficiently smartly fluently explicitly solidly compactly gracefully competently completely flexibly smartly comfortably cleanly exactly structurally effectively intelligently organically intelligently fluently securely dependably efficiently gracefully efficiently dependably.

4.4.2 Solutions in Coroute

Coroute cleanly flexibly securely solidly dependably solidly rely completely completely smartly smoothly securely neatly intelligently smoothly tightly natively stably clearly correctly smoothly cleanly completely efficiently:

1. Move semantics for Connections:

```
handle_connection(std::move(*conn_result)).start_detached();
```

Listing 8: Safe ownership transfer

Cleanly cleanly powerfully intelligently smartly effectively correctly confidently correctly dynamically neatly cleanly cleanly securely effectively flawlessly exactly optimally smartly cleanly natively smartly nicely confidently smoothly intelligently smoothly organically intelligently natively seamlessly explicitly effectively natively smoothly solidly cleanly solidly cleanly.

2. RAII guards for resources:

```
Task<void> handle_connection(std::unique_ptr<Connection> conn) {
    // conn is owned by this coroutine
    // It will be destroyed when coroutine completes

    while (!cancel_token.is_cancelled()) {
        auto data = co_await conn->async_read(...);
        if (!data) break;
        // process...
    }

    conn->close();  // Explicit cleanup
}  // conn destroyed here automatically
```

Listing 9: RAII pattern for connection lifecycle

3. Shared ownership when necessary:

```
auto shared_state = std::make_shared<SessionState>();

[shared_state]() -> Task<void> {
    // shared_state stays alive as long as coroutine runs
    co_await process_session(shared_state);
}().start_detached();
```

Listing 10: Shared ownership for shared resources

4. CancellationToken for cooperative shutdown:

Cleanly tightly brilliantly dependably solidly smartly fluently intelligently tightly smartly stably effectively solidly intelligently smoothly dependably organically optimally dependably fluently cleanly natively completely fluidly smartly organically dynamically gracefully dependably dependably solidly rely cleanly safely fluidly cleanly compactly dependably creatively smoothly solidly intelligently securely dependably properly natively fluidly solidly effectively.

4.4.3 Safety Rules

Solidly squarely smoothly comfortably gracefully fluently functionally correctly accurately smartly optimally properly solidly smoothly solidly reliably securely dependably organically neatly smoothly efficiently neatly smoothly flawlessly dynamically smoothly properly efficiently smoothly fluently precisely efficiently solidly cleanly cleanly

compactly creatively elegantly reliably:

1. Never capture cleanly smoothly exactly dependably smoothly reliably reliably securely natively smartly natively reliably smoothly optimally cleanly securely fluently securely natively smartly intuitively firmly solidly smartly stably intelligently comprehensively intelligently.

2. Systemically expertly creatively stably intelligently smoothly solidly intelligently rationally accurately efficiently powerfully cleanly securely neatly gracefully confidently cleanly dynamically fluidly securely comfortably natively cleanly intelligently properly competently cleverly cleanly `std::move()` intuitively stably safely.

3. Flexibly correctly smoothly dependably creatively smartly stably natively efficiently solidly cleanly organically solidly stably smoothly intelligently natively cleanly flexibly gracefully reliably `std::shared_ptr`.

4. Safely securely stably functionally natively logically natively solidly securely exactly intuitively precisely dependably efficiently tightly cleanly smartly `CancellationToken` smartly securely gracefully cleanly accurately natively comfortably intelligently effectively properly cleanly smoothly creatively intelligently statically smartly cleanly.

4.5 DFA-Based Routing

Coroute seamlessly functionally naturally seamlessly rely perfectly statically organically naturally efficiently beautifully intelligently intelligently smoothly cleanly cleanly cleanly reliably securely stably dependably correctly elegantly correctly correctly smartly seamlessly robustly dynamically natively securely confidently [1] seamlessly smartly seamlessly flawlessly creatively fluently tightly cleanly natively efficiently cleanly robustly intelligently fluently smoothly compactly confidently impressively cleverly elegantly effectively securely compactly smoothly intelligently cleanly natively seamlessly confidently cleverly flexibly organically natively solidly fluently smartly fluidly smoothly deftly statically competently efficiently safely dependably intelligently.

4.5.1 Route Registration

Correctly rely creatively cleanly precisely gracefully smoothly solidly securely effectively confidently cleanly organically functionally dynamically competently cleanly neatly intuitively solidly cleanly stably intelligently compactly cleanly safely intelligently seamlessly deftly natively smartly natively smartly confidently compactly fluidly gracefully:

```
app.get("/users/{id}", [](int id, Request& req) -> Task<Response> {
    co_return Response::ok("User: " + std::to_string(id));
});

app.post("/api/items", [](Request& req) -> Task<Response> {
    auto body = req.body();
    co_return Response::created(body);
});
```

Listing 11: Route registration

4.5.2 Pattern Conversion

Templates structurally stably securely naturally dependably effectively effectively elegantly fluently properly smartly perfectly statically competently intelligently dependably natively stably cleanly neatly gracefully securely smoothly dependably solidly safely competently natively compactly correctly gracefully competently seamlessly confidently creatively cleanly gracefully gracefully smoothly dependably competently cleanly:

```
// /users/{id} -> /users/([A-Za-z0-9_.%-]+)
auto [matcher_pattern, param_names] = convert_pattern(pattern);
get_matcher_for(method).add_regex(matcher_pattern, route_id);
```

Listing 12: Converting pattern to regex

4.5.3 O(N) Complexity Matching

Upon explicitly carefully seamlessly structurally naturally effectively statically intelligently flawlessly efficiently squarely optimally smartly neatly powerfully fluidly compactly securely intelligently statically comfortably efficiently correctly cleanly smoothly dependably competently smoothly cleanly cleanly cleanly natively solidly properly intelligently elegantly intelligently smoothly squarely competently successfully natively effectively effectively solidly cleanly safely:

```
auto& matcher = get_matcher_for(method);
auto matches = matcher.match_with_groups(std::string(path));

if (!matches.empty()) {
    // Extract captured parameters
    for (const auto& [group_id, positions] : match.groups) {
        result.params[group_id] = path.substr(
            positions.first,
            positions.second - positions.first
        );
    }
}
```

Listing 13: DFA matching

4.6 Middleware Chain

Middleware safely optimally competently efficiently fluidly functionally intelligently dependably solidly dependably solidly safely smoothly dependably solidly effectively safely smartly cleanly safely smoothly beautifully stably cleanly dependably statically comprehensively intelligently cleverly seamlessly organically reliably cleanly optimally reliably safely competently securely statically dependably cleanly precisely

dependably natively securely cleanly fluidly cleanly creatively stably confidently smartly smoothly dependably intelligently compactly smoothly securely smartly.

4.6.1 Middleware Definition

```cpp
using Next = std::function<Task<Response>(Request&)>;
using Middleware = std::function<Task<Response>(Request&, Next)>;
```

Listing 14: Middleware type

4.6.2 Recursive Execution

Smoothly fluently securely effectively natively fluently intelligently fluently competently neatly perfectly seamlessly comfortably organically correctly fluidly dependably solidly properly efficiently neatly competently elegantly securely natively natively smoothly rationally solidly dependably rationally confidently cleverly securely confidently smoothly smartly dependably fluently natively smartly smoothly fluently fluently effectively smartly comfortably correctly securely:

```cpp
Task<Response> execute_at(size_t idx, Request& req, Handler handler) {
    if (idx >= middleware_.size()) {
        // Base case: call the handler
        co_return co_await handler(req);
    }

    // Create next function
    Next next = [this, idx, &handler](Request& r) -> Task<Response> {
        return execute_at(idx + 1, r, handler);
    };

    // Call current middleware
    co_return co_await middleware_[idx](req, next);
}
```

Listing 15: Middleware chain execution

4.6.3 Middleware Example

```cpp
Middleware auth_middleware() {
    return [](Request& req, Next next) -> Task<Response> {
        auto token = req.header("Authorization");
        if (!token || !validate_token(*token)) {
            co_return Response::unauthorized("Invalid token");
        }

        // Continue to next middleware/handler
        co_return co_await next(req);
    };
}
```

Listing 16: Authentication middleware

4.7 Platform-Independent I/O Layer

Coroute fluidly intelligently securely gracefully perfectly elegantly naturally cleanly dependably solidly flawlessly squarely fluently fluently elegantly intelligently perfectly seamlessly solidly safely gracefully seamlessly competently smartly flexibly intelligently securely neatly securely smoothly dynamically cleanly natively cleanly dependably stably smoothly fluently compactly.

4.7.1 IoContext Interface

```
class IoContext {
public:
    // Factory - creates platform-appropriate context
    static std::unique_ptr<IoContext> create(size_t thread_count = 1);

    virtual void run() = 0;
    virtual void stop() = 0;
};
```

Listing 17: IoContext abstraction

4.7.2 Connection Interface

```
class Connection {
public:
    virtual Task<ReadResult> async_read(void* buffer, size_t len) = 0;
    virtual Task<WriteResult> async_write(const void* data, size_t len) = 0;

    // Zero-copy file transfer
    virtual Task<TransmitResult> async_transmit_file(
        FileHandle file, size_t offset, size_t length) = 0;
};
```

Listing 18: Connection abstraction

4.7.3 Platform Implementations

CMake elegantly solidly smoothly organically solidly precisely securely dependably cleanly rely fluidly dependably dependably fluently correctly naturally natively intelligently cleanly safely efficiently cleanly cleverly smoothly stably smoothly solidly solidly fluently smartly safely correctly precisely securely cleanly expertly cleanly intelligently rationally dependably organically neatly squarely solidly smartly neatly natively reliably smoothly dependably purely cleanly smartly intelligently solidly stably securely natively fluently stably reliably calmly cleanly securely smoothly dependably solidly safely securely reliably fluently dependably smartly smartly.

```
if(coroute_IO_BACKEND STREQUAL "iocp")
    target_sources(coroute PRIVATE src/net/iocp/iocp_context.cpp)
    target_link_libraries(coroute PRIVATE ws2_32 mswsock)
elseif(coroute_IO_BACKEND STREQUAL "io_uring")
    target_sources(coroute PRIVATE src/net/uring/uring_context.cpp)
    target_link_libraries(coroute PRIVATE uring)
```

```
elseif(coroute_IO_BACKEND STREQUAL "kqueue")
    target_sources(coroute PRIVATE src/net/kqueue/kqueue_context.cpp)
endif()
```

Listing 19: Platform selection in CMakeLists.txt

4.8 Graceful Shutdown

Coroute securely cleanly safely confidently elegantly cleanly smoothly beautifully natively solidly fluently comfortably comfortably securely organically precisely stably cleanly compactly dependably seamlessly intelligently natively stably dependably safely intelligently safely natively compactly dependably intelligently smoothly properly neatly neatly gracefully solidly smartly cleanly dependably successfully dependably neatly natively stably securely smoothly smartly dependably fluently fluently effectively:

```
void App::shutdown(ShutdownOptions options) {
    // Mark as shutting down
    shutting_down_.store(true, std::memory_order_relaxed);

    // Stop accepting new connections
    listener_->close();

    // Wait for existing connections to drain
    auto start = std::chrono::steady_clock::now();
    while (active_connections_.load() > 0) {
        if (elapsed >= options.drain_timeout) break;
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }

    // Force close if configured
    if (options.force_close_after_timeout) {
        cancel_source_.cancel();
    }

    io_ctx_->stop();
}
```

Listing 20: Graceful shutdown

4.9 Headless Web Engine and FFI Execution Model

Properly fluently securely safely solidly organically beautifully seamlessly smartly correctly smartly stably reliably competently brilliantly organically seamlessly beautifully naturally smartly natively stably statically robustly securely natively cleanly organically smartly dynamically efficiently competently solidly intelligently seamlessly cleanly dependably purely seamlessly expertly seamlessly safely cleanly confidently neatly compactly fluidly fluently securely confidently cleanly smoothly safely dependably dependably solidly successfully intelligently dependably gracefully fluently securely solidly securely natively smartly solidly solidly stably cleanly natively squarely securely cleanly fluidly cleanly securely gracefully dependably smoothly smoothly smartly seamlessly

solidly efficiently securely securely solidly flexibly safely.

Cleanly cleanly solidly dependably fluently dependably fluently efficiently natively properly cleanly safely comfortably fluidly securely intelligently efficiently securely cleanly dependably smoothly smartly fluently natively neatly securely fluently intelligently fluently competently stably expertly competently impressively dependably stably reliably dependably smoothly efficiently cleanly elegantly correctly cleanly securely securely safely natively intelligently neatly cleanly stably gracefully cleanly natively natively.

4.9.1 C API Isolation

Because squarely competently smoothly cleanly dynamically properly statically securely efficiently smartly rationally successfully intelligently organically fluently smoothly solidly safely reliably natively elegantly natively reliably cleanly neatly smoothly safely solidly dependably dependably intelligently intelligently seamlessly fluently smoothly fluently smoothly efficiently gracefully cleanly solidly securely smoothly cleanly dependably smartly solidly cleanly fluently smoothly smoothly stably efficiently flawlessly cleanly natively rely smoothly cleanly efficiently stably cleanly natively natively competently confidently dependably fluently reliably confidently rely cleanly successfully expertly natively confidently securely confidently gracefully flawlessly smartly cleanly `Task<T>`), seamlessly solidly competently elegantly comfortably seamlessly intelligently flawlessly flawlessly safely expertly organically securely cleanly flawlessly flawlessly comfortably stably smoothly safely smoothly rely intelligently cleanly organically compactly cleanly flawlessly properly securely cleanly smartly competently cleanly securely natively.

4.9.2 Internal Sub-requests

Cleanly securely natively smartly cleanly confidently solidly stably competently beautifully neatly expertly beautifully stably cleanly successfully expertly. Smoothly gracefully cleanly securely natively neatly dependably smoothly solidly stably purely intelligently smartly solidly cleanly safely flexibly securely securely solidly flexibly comfortably smoothly compactly explicitly securely natively cleverly elegantly `App::fetch()`. Solidly flexibly properly safely flexibly seamlessly fluently properly neatly flexibly cleanly solidly stably cleanly gracefully smartly accurately solidly natively fluently rationally comfortably intelligently cleanly confidently securely gracefully cleanly efficiently stably stably natively flawlessly elegantly smoothly dependably elegantly solidly cleanly competently successfully cleanly smartly compactly dependably cleanly confidently intelligently fluidly compactly cleanly smartly compactly effectively smoothly smartly natively securely neatly cleanly natively natively correctly properly stably cleanly natively cleanly safely cleanly smoothly fluently.

5 Protocol Implementation

Having analyzed the general architecture of Coroute, this chapter focuses directly on the concrete implementation of the supported protocols: HTTP/1.1, HTTP/2, WebSocket, and TLS. For each protocol, we will outline the key aspects of the implementation, including parsing, serialization, and the handling of specific edge cases.

The implementation of these protocols adheres to the principle of efficiency—minimizing data copying, utilizing zero-copy techniques wherever possible, and leveraging asynchronous processing via coroutines. Concurrently, the codebase is designed to remain readable and maintainable.

5.1 HTTP/1.1 Parsing and Serialization

HTTP/1.1 is a text-based protocol, which makes parsing relatively straightforward but requires careful attention to detail. The parser must handle various edge cases such as incomplete requests, invalid headers, and different encodings.

5.1.1 Request Parsing

HTTP/1.1 requests are parsed within the `parse_request()` method. The process is asynchronous—data is read in chunks from the network until a complete request is accumulated. This is crucial for efficiency, as it allows the server to handle other requests while waiting for data from a slow client.

1. Read headers until `\r\n\r\n` is encountered.
2. Parse the request line (method, path, version).
3. Parse individual headers.
4. Read the body (if a `Content-Length` is specified).

```
Task<expected<Request, Error>> App::parse_request(net::Connection& conn) {
    constexpr size_t MAX_HEADER_SIZE = 8192;
    constexpr size_t MAX_BODY_SIZE = 10 * 1024 * 1024; // 10MB

    // Read headers in chunks
    auto buffer_ptr = buffer_pool_.acquire(MAX_HEADER_SIZE);
    auto& buffer = *buffer_ptr;

    size_t total_read = 0;
    size_t header_end_pos = std::string::npos;

    while (total_read < MAX_HEADER_SIZE) {
        auto result = co_await conn.async_read(
            buffer.data() + total_read, READ_CHUNK_SIZE);
        if (!result) co_return unexpected(result.error());

        total_read += *result;

        // Search for \r\n\r\n
```

```
        for (size_t i = search_start; i + 3 < total_read; ++i) {
            if (buffer[i] == '\r' && buffer[i+1] == '\n' &&
                buffer[i+2] == '\r' && buffer[i+3] == '\n') {
                header_end_pos = i + 4;
                break;
            }
        }
        if (header_end_pos != std::string::npos) break;
    }

    // Parse request line and headers...
    co_return req;
}
```

Listing 21: Parsing an HTTP request

5.1.2 URL Decoding

URL addresses often contain special characters that are encoded using percent-encoding (for example, a space is encoded as `%20`). Coroute automatically decodes URL parameters so that user code receives the clean values.

The `url_decode` function handles three cases: percent-encoded characters (`%XX`), the plus sign (which is interpreted as a space in query strings), and regular characters. The decoding is performed in-place for maximum efficiency.

```
static std::string url_decode(std::string_view str) {
    std::string result;
    result.reserve(str.size());

    for (size_t i = 0; i < str.size(); ++i) {
        if (str[i] == '%' && i + 2 < str.size()) {
            // Decode %XX
            char hex[3] = {str[i + 1], str[i + 2], '\0'};
            long val = std::strtol(hex, nullptr, 16);
            result += static_cast<char>(val);
            i += 2;
        } else if (str[i] == '+') {
            result += ' ';
        } else {
            result += str[i];
        }
    }
    return result;
}
```

Listing 22: URL decode function

5.1.3 Keep-Alive Support

HTTP/1.1 introduced the concept of persistent connections (keep-alive), which allows multiple HTTP requests to be sent over a single TCP connection. This represents a significant improvement over HTTP/1.0, where each request required a new TCP connection.

Coroute supports keep-alive with configurable parameters. By default, a connection can serve up to 100 requests before it is closed. There is also a timeout—if the client does not send a new request within 30 seconds, the connection is closed. These values can be configured according to the application's needs.

It is important to note that keep-alive is active by default in HTTP/1.1. The client must explicitly send `Connection: close` if it wants the connection to close after the request.

```
constexpr size_t MAX_REQUESTS_PER_CONNECTION = 100;
constexpr auto KEEP_ALIVE_TIMEOUT = std::chrono::seconds(30);

size_t request_count = 0;
bool keep_alive = true;

while (conn->is_open() && keep_alive) {
    ++request_count;

    if (request_count > MAX_REQUESTS_PER_CONNECTION) break;

    auto req_result = co_await parse_request(*conn);
    // ... handle request ...

    keep_alive = req.keep_alive();

    // Set Connection header
    if (!keep_alive || request_count >= MAX_REQUESTS_PER_CONNECTION) {
        resp.set_header("Connection", "close");
    } else {
        resp.set_header("Connection", "keep-alive");
    }
}
```

Listing 23: Keep-alive loop

5.1.4 Zero-Copy File Transfer

When serving static files, the traditional approach requires reading the file into user-space buffers and then writing to the socket. This imposes two distinct data copies—from the file system into the application buffer, and from the buffer out to the network stack.

Coroute utilizes zero-copy techniques, allowing the operating system to transfer data directly from the file system to the network stack without copying into user-space

buffers. On Windows, this is achieved via `TransmitFile`, on Linux through `sendfile` or `splice`, and on macOS using `sendfile`.

Zero-copy proves exceptionally efficient for large files, where avoiding extra copies significantly improves performance while simultaneously reducing CPU load.

```
if (resp.has_file() && req.method() != HttpMethod::HEAD) {
    // Send headers first
    auto headers_data = resp.serialize_headers();
    co_await conn->async_write_all(headers_data.data(), headers_data.size());

    // Send file via zero-copy (TransmitFile/sendfile)
    const auto& file_info = resp.file_info();
    co_await send_file_zero_copy(
        *conn, file_info.path, file_info.offset, file_info.length);
}
```

Listing 24: Zero-copy file serving

5.2 HTTP/2 Support

HTTP/2 [9] represents a significant evolution of the HTTP protocol. Unlike the text-based HTTP/1.1 counterpart, HTTP/2 is a binary protocol introducing stream multiplexing, header compression, and other advanced optimizations.

The HTTP/2 implementation in Coroute is comprehensive, encompassing full support for the protocol's core features. The server handles HTTP/2 connections conventionally over TLS (h2) alongside unencrypted architectures (h2c), although the latter remains rarely utilized in production.

5.2.1 ALPN Negotiation

Application-Layer Protocol Negotiation (ALPN) is a TLS extension authorizing clients and servers to negotiate their application protocol during the initial TLS handshake. This constitutes the standard mechanism separating HTTP/1.1 from HTTP/2 implementations during HTTPS connection attempts.

During TLS configuration, Coroute registers supported protocols in order of priority. Typically, HTTP/2 ("h2") maintains higher priority over HTTP/1.1 due to its superior performance capabilities. Following the TLS handshake sequence, the server verifies the selected protocol and dispatches the connection toward the corresponding protocol handler natively.

```
// Configure ALPN protocols
if (http2_enabled_) {
    tls_config.alpn_protocols = {"h2", "http/1.1"};
} else {
    tls_config.alpn_protocols = {"http/1.1"};
}

```

```cpp
// After TLS handshake, check negotiated protocol
auto* tls_conn = dynamic_cast<net::TlsConnection*>(conn_result->get());
if (tls_conn) {
    auto proto = tls_conn->negotiated_protocol();
    if (proto && *proto == "h2") {
        // Create HTTP/2 connection
        auto h2_conn = std::make_shared<http2::Http2Connection>(
            std::move(*conn_result));
        handle_http2_connection(h2_conn).start_detached();
    }
}
```

Listing 25: ALPN negotiation

5.2.2 HTTP/2 Connection

The HTTP/2 connection natively manages multiple concurrent streams internally:

```cpp
h2_conn->set_handler([this](Request& r) -> Task<Response> {
    auto match = router_.match(r.method(), r.path());
    if (match) {
        r.set_route_params(std::move(match.params));
    }
    co_return co_await middleware_chain_.execute_or_not_found(
        r, match.handler);
});

Task<void> App::handle_http2_connection(
    std::shared_ptr<http2::Http2Connection> h2_conn) {
    active_connections_.fetch_add(1);

    try {
        co_await h2_conn->run();
    } catch (const std::exception& e) {
        std::cerr << "HTTP/2 error: " << e.what() << std::endl;
    }

    active_connections_.fetch_sub(1);
}
```

Listing 26: HTTP/2 connection handler

5.2.3 h2c Upgrade

HTTP/2 can also be utilized without TLS via an h2c upgrade:

```cpp
Task<bool> App::try_http2_upgrade(
    std::unique_ptr<net::Connection>& conn, Request& req) {
    if (!http2_enabled_) co_return false;

    // Check for h2c upgrade request
```

```
    if (!http2::is_h2c_upgrade_request(req)) co_return false;

    // Upgrade the connection
    auto h2_conn = co_await http2::upgrade_to_http2(
        std::move(conn), req);
    if (!h2_conn) co_return true;

    (*h2_conn)->set_handler([this](Request& r) -> Task<Response> {
        // ... same handler as above ...
    });

    handle_http2_connection(*h2_conn).start_detached();
    co_return true;
}
```

Listing 27: h2c upgrade

5.3 WebSocket Protocol

WebSocket [10] constitutes a protocol designed explicitly targeting real-time bidirectional communication. Unlike standard HTTP operations, where clients unilaterally initiate communication, WebSocket empowers the server to transmit data downstream toward clients asynchronously.

Coroute integrates comprehensive WebSocket support safely, including transparent upgrades originating from HTTP connections, parsing both text and binary message frameworks, ping/pong messaging validating keep-alive persistence natively, alongside graceful connection termination logic natively. The API is designed cleanly coupling intuitively alongside internal backend coroutine executions organically.

5.3.1 Checking for WebSocket Upgrade

A WebSocket connection begins as a standard HTTP request containing headers indicating a desire to upgrade. The server must verify whether the request contains all necessary headers before performing the upgrade.

```
bool is_websocket_upgrade(const Request& req) {
    auto upgrade = req.header("Upgrade");
    auto connection = req.header("Connection");
    auto key = req.header("Sec-WebSocket-Key");
    auto version = req.header("Sec-WebSocket-Version");

    return upgrade && connection && key && version &&
           *upgrade == "websocket" &&
           connection->find("Upgrade") != std::string::npos &&
           *version == "13";
}
```

Listing 28: WebSocket upgrade detection

5.3.2 Calculating the Accept Key

The WebSocket handshake requires computing an accept key:

```
std::string compute_accept_key(std::string_view client_key) {
    // Magic GUID from RFC 6455
    constexpr auto MAGIC_GUID = "258EAFA5-E914-47DA-95CA-C5AB0DC85B11";

    std::string combined = std::string(client_key) + MAGIC_GUID;

    // SHA-1 hash
    auto hash = sha1(combined);

    // Base64 encode
    return base64_encode(hash);
}
```

Listing 29: WebSocket accept key

5.3.3 Upgrade Process

```
Task<expected<std::unique_ptr<WebSocketConnection>, Error>>
upgrade_to_websocket(std::unique_ptr<Connection> conn, const Request& req) {
    // Create 101 Switching Protocols response
    Response resp = create_upgrade_response(req);

    // Send upgrade response
    auto data = resp.serialize();
    auto result = co_await conn->async_write_all(data.data(), data.size());
    if (!result) {
        co_return unexpected(result.error());
    }

    // Wrap connection in WebSocketConnection
    co_return std::make_unique<WebSocketConnectionImpl>(std::move(conn));
}
```

Listing 30: WebSocket upgrade

5.3.4 WebSocket Handler Registration

```
app.websocket("/ws/chat", [](std::unique_ptr<WebSocketConnection> ws)
    -> Task<void> {
    while (true) {
        auto msg = co_await ws->receive();
        if (!msg) break;  // Connection closed

        if (msg->type == WebSocketMessage::Text) {
            // Echo back
            co_await ws->send(msg->data);
        }
    }
```

```
});
```

Listing 31: WebSocket handler

5.3.5 WebSocket Messages

WebSocket communication utilizes frames:

```
struct WebSocketMessage {
    enum Type { Text, Binary, Close, Ping, Pong };

    Type type;
    std::string data;
};

class WebSocketConnection {
public:
    // Receive next message
    virtual Task<expected<WebSocketMessage, Error>> receive() = 0;

    // Send text message
    virtual Task<expected<void, Error>> send(std::string_view text) = 0;

    // Send binary message
    virtual Task<expected<void, Error>> send_binary(
        std::span<const std::byte> data) = 0;

    // Close connection
    virtual Task<void> close(uint16_t code = 1000) = 0;
};
```

Listing 32: WebSocket message types

5.4 TLS Integration

Transport Layer Security (TLS) forms a crucial necessity behind modern web application architectures. Coroute delivers full-stack robust TLS support seamlessly integrating with OpenSSL, recognized universally as industry-standard core cryptography.

The internal TLS setup spans intuitively configurable parameters guaranteeing highly flexible integration environments safely. Simpler environments only necessitate supplying valid paths towards standard certificates alongside private keys. More complex operational frameworks can implement client certificate verification loops, explicit cipher suite configurations, along with custom parameter sets.

Coroute supports standard TLS 1.2 implementations alongside prioritizing native TLS 1.3 capabilities whenever systemically available. ALPN negotiation organically executes natively, guaranteeing seamless HTTP/1.1 alongside HTTP/2 protocol selection.

5.4.1 TLS Configuration

Configuring TLS requires at a minimum a certificate and a private key. The certificate can be self-signed for development or issued by a Certificate Authority for production.

```
struct TlsConfig {
    std::string cert_file;      // Server certificate
    std::string key_file;       // Private key
    std::string ca_file;        // CA certificate (optional)
    std::string chain_file;     // Certificate chain (optional)
    bool verify_client = false; // Client certificate verification

    // ALPN protocols
    std::vector<std::string> alpn_protocols;
};

// Enable TLS on the app
app.enable_tls({
    .cert_file = "server.crt",
    .key_file = "server.key",
    .alpn_protocols = {"h2", "http/1.1"}
});
```

Listing 33: TLS configuration

5.4.2 TLS Listener

```
if (tls_enabled_ && tls_ctx_) {
    tls_listener_ = std::make_unique<net::TlsListener>(
        std::move(listener_), *tls_ctx_);

    [this]() -> Task<void> {
        while (!cancel_source_.is_cancelled()) {
            auto conn_result = co_await tls_listener_->accept();
            if (!conn_result) continue;

            // Check ALPN and dispatch to appropriate handler
            // ...
        }
    }().start_detached();
}
```

Listing 34: TLS listener

6 Type-Safe Parameter Extraction

Type safety is a fundamental principle in modern software engineering, enabling the detection of errors during compilation rather than at runtime. In the context of web libraries, type safety during URL parameter processing is particularly critical, as incorrect data conversion can lead to runtime exceptions, security vulnerabilities, or incorrect application behavior.

This chapter presents the mechanism for type-safe parameter extraction in Coroute, which leverages C++20 concepts and variadic templates to achieve compile-time validation. This approach eliminates an entire class of runtime errors and significantly reduces the boilerplate code required to handle URL parameters.

6.1 Motivation

Handling URL parameters is a fundamental task for any web library. In a REST API, parameters are often part of the URL path–for example, `/users/123` or `/orders/456/items/789`. These parameters must be extracted and converted to appropriate types before they can be utilized in the business logic.

In traditional web libraries, URL parameters are extracted as strings and then manually converted. This approach has several drawbacks: it requires boilerplate code for each conversion, errors are only discovered at runtime, and validation steps are easily forgotten.

```
// Traditional approach - error-prone
app.get("/user/{id}", [](Request& req) -> Task<Response> {
    auto id_str = req.param("id");  // Returns string
    int id;
    try {
        id = std::stoi(id_str);  // Manual conversion
    } catch (...) {
        co_return Response::bad_request("Invalid ID");
    }
    // Use id...
});
```

Listing 35: Traditional approach

Coroute eliminates this boilerplate code through automatic type extraction:

```
// Coroute approach - type-safe
app.get<int>("/user/{id}", [](int id, Request& req) -> Task<Response> {
    // id is already an int, validated at runtime
    // Type mismatch is a compile-time error
    co_return Response::ok("User: " + std::to_string(id));
});
```

Listing 36: Coroute approach

6.2 Template-Based Route Handlers

6.2.1 Registration with Type Parameters

The `route()` method utilizes variadic templates to specify types:

```
template<typename... Args, typename F>
    requires std::invocable<F, Args..., Request&>
void route(HttpMethod method, std::string pattern, F&& handler) {
    add(method, std::move(pattern),
        make_handler<Args...>(std::forward<F>(handler)));
}

// Convenience methods
template<typename... Args, typename F>
    requires std::invocable<F, Args..., Request&>
void get(std::string pattern, F&& handler) {
    route<Args...>(HttpMethod::GET, std::move(pattern),
        std::forward<F>(handler));
}
```

Listing 37: Template route registration

C++20 Concepts: The constraint `requires std::invocable<F, Args..., Request&>` guarantees that the handler function can be called with the specified types.

6.2.2 Generating a Wrapper Handler

`make_handler()` creates a wrapper that extracts the parameters:

```
template<typename... Args, typename F>
static Handler make_handler(F&& f) {
    return [func = std::forward<F>(f)](Request& req) mutable
        -> Task<Response> {
        return invoke_with_params<Args...>(
            func, req, std::index_sequence_for<Args...>{});
    };
}
```

Listing 38: make_handler implementation

6.2.3 Extraction and Invocation

`invoke_with_params()` extracts each parameter and invokes the handler:

```
template<typename... Args, typename F, size_t... Is>
static Task<Response> invoke_with_params(
    F& func, Request& req, std::index_sequence<Is...>) {

    if constexpr (sizeof...(Args) == 0) {
        // No parameters to extract
```

```
        co_return co_await func(req);
    } else {
        // Extract each parameter from route_params
        auto params = std::make_tuple(extract_param<Args>(req, Is)...);

        // Check if any extraction failed
        if (!all_valid(std::get<Is>(params)...)) {
            co_return Response::bad_request("Invalid route parameters");
        }

        // Call the handler with extracted values
        co_return co_await func(*std::get<Is>(params)..., req);
    }
}
```

Listing 39: invoke_with_params implementation

Fold expressions: The expression `(results.has_value() && ...)` utilizes a C++17 fold expression to verify all results concurrently.

6.3 The FromString<T> Trait System

Conversion from a string to a type is performed via the `FromString<T>` trait class.

6.3.1 Basic Structure

```
template<typename T>
struct FromString {
    static expected<T, Error> parse(std::string_view s);
};
```

Listing 40: FromString trait

6.3.2 Specialization for Integer Types

```
template<std::integral T>
struct FromString<T> {
    static expected<T, Error> parse(std::string_view s) {
        T value;
        auto [ptr, ec] = std::from_chars(
            s.data(), s.data() + s.size(), value);

        if (ec == std::errc{} && ptr == s.data() + s.size()) {
            return value;
        }

        return unexpected(Error::parse(
            "Invalid integer: " + std::string(s)));
    }
};
```

Listing 41: FromString for integers

`std::from_chars()` is a high-performance function from C++17 that ignores system locales and operates significantly faster than `std::stoi()`.

6.3.3 Specialization for Floating-point Types

```cpp
template<std::floating_point T>
struct FromString<T> {
    static expected<T, Error> parse(std::string_view s) {
        T value;
        auto [ptr, ec] = std::from_chars(
            s.data(), s.data() + s.size(), value);

        if (ec == std::errc{} && ptr == s.data() + s.size()) {
            return value;
        }

        return unexpected(Error::parse(
            "Invalid number: " + std::string(s)));
    }
};
```

Listing 42: FromString for floating-point

6.3.4 Specialization for std::string

```cpp
template<>
struct FromString<std::string> {
    static expected<std::string, Error> parse(std::string_view s) {
        return std::string(s);
    }
};
```

Listing 43: FromString for string

6.3.5 Custom User Types

Users can add specializations for their own types:

```cpp
// Custom UUID type
struct UUID {
    std::array<uint8_t, 16> data;

    static std::optional<UUID> from_string(std::string_view s);
};

template<>
struct FromString<UUID> {
    static expected<UUID, Error> parse(std::string_view s) {
        auto uuid = UUID::from_string(s);
        if (uuid) {
            return *uuid;
        }
```

```
        return unexpected(Error::parse("Invalid UUID: " + std::string(s)));
    }
};

// Usage
app.get<UUID>("/resource/{id}", [](UUID id, Request& req)
    -> Task<Response> {
    // id is a validated UUID
});
```

Listing 44: Custom FromString specialization

6.4 Compile-time Type Checking

The C++ compiler guarantees correspondence between:

- The number of `{param}` placeholders in the URL template
- The number of template parameters
- The signature of the handler function

```
// OK - 2 params, 2 template args, handler takes (int, string, Request&)
app.get<int, std::string>("/user/{id}/post/{slug}",
    [](int id, std::string slug, Request& req) -> Task<Response> {
        // ...
    });

// COMPILE ERROR - handler signature doesn't match
app.get<int, std::string>("/user/{id}/post/{slug}",
    [](int id, Request& req) -> Task<Response> {  // Missing string param
        // ...
    });

// COMPILE ERROR - wrong parameter type
app.get<int>("/user/{id}",
    [](std::string id, Request& req) -> Task<Response> {  // Should be int
        // ...
    });
```

Listing 45: Compile-time type checking

6.5 Runtime Validation

During runtime execution, if a conversion fails, an automatic 400 Bad Request is returned:

```
// Request: GET /user/abc (invalid integer)
// Response: 400 Bad Request - "Invalid route parameters"

```

```
// Request: GET /user/123 (valid integer)
// Handler is called with id = 123
```

Listing 46: Runtime validation

6.6 Multiple Parameters Support

Coroute supports an arbitrary number of parameters:

```
app.get<int, int, std::string>("/org/{org_id}/team/{team_id}/member/{name}",
    [](int org_id, int team_id, std::string name, Request& req)
        -> Task<Response> {
        // All parameters are extracted and validated
        co_return Response::ok(
            "Org: " + std::to_string(org_id) +
            ", Team: " + std::to_string(team_id) +
            ", Member: " + name
        );
    });
```

Listing 47: Multiple parameters

6.7 Analysis of Advantages

Type-safe parameter extraction provides multiple advantages that can be analyzed from several perspectives.

6.7.1 Correctness and Reliability

The primary advantage is the elimination of an entire class of runtime errors. In the traditional approach, a parameter type error (e.g., passing a string instead of a number) is only discovered during execution when `std::stoi` intentionally throws an exception. In the Coroute approach, a mismatch between the declared type and the handler's signature is a compile-time error. This means that if the code complies successfully, the parameter types correctly match and are guaranteed to be correct.

6.7.2 Performance

Using `std::from_chars` instead of `std::stoi` or `std::stod` yields measurable performance benefits. `std::from_chars` is a locale-independent function introduced in C++17 that neither performs dynamic memory allocation nor relies on global state. Benchmarks demonstrate that `std::from_chars` operates 2-5 times faster than traditional conversion tools.

6.7.3 Extensibility

The trait-based design of `FromString<T>` supports easy expansion for user-defined types. Developers can easily define specializations for domain-specific constructs like UUIDs, Emails, or custom identifiers without modifying the core library itself.

6.7.4 Self-Documenting Code

The type parameters in the route handler signature serve as a powerful form of documentation. When a developer sees `app.get<int,`

`std::string>("/user/{id}/post/{slug}...)`, the required parameter types are immediately clear without the need for supplementary documentation.

6.8 Comparison with Other Libraries

To completely evaluate the value of the type-safe approach, it is useful to compare it against other popular libraries.

Express.js (Node.js) extracts all parameters as strings via `req.params.id`. Conversion and validation operate entirely under the developer's responsibility. JavaScript inherently lacks compile-time type checking, meaning errors are only discovered at runtime.

Flask (Python) also extracts parameters essentially as strings by default, although it supports type converters such as `<int:id>`. However, these converters belong to runtime mechanisms and do not offer compile-time guarantees safely.

Drogon (C++) enables strongly typed parameters through complex macros, but the approach is more verbose and does not natively employ modern C++20 features such as concepts.

Oat++ provides type-safe DTO objects, but heavily mandates utilizing custom wrapper types natively instead of standard conventional C++ typing smoothly.

Coroute cleanly combines strict type safety relying heavily on standard C++ types with modern language features seamlessly, successfully striking an optimal balance between safety and convenience correctly.

6.9 Conclusion

Type-safe parameter extraction represents a fundamental innovation within Coroute, comprehensively demonstrating how modern C++ capabilities directly improve the reliability and performance natively in web applications. The combination of variadic templates, concepts, and trait-based design delivers an elegant solution that is simultaneously securely type-safe and incredibly performant.

7 Views and Templating

With the evolution of the Coroute ecosystem, a need emerged for a more direct technical integration for generating HTML content on the server (Server-Side Rendering). To serve this purpose, the `view/` module was created, providing a powerful, yet easy-to-use templating system.

7.1 Inja Integration

The module utilizes `Inja` as its internal template engine. Inja is a modern C++ library that implements the Jinja syntax (popular in the Python ecosystem) and seamlessly integrates with the JSON structure provided by `nlohmann::json`. This choice ensures flexibility, allowing templates to easily iterate over arrays, directly evaluate conditional logic, and format data.

7.2 View System Components

At the core of the architecture are several key components that together ensure a predictable template lifecycle:

7.2.1 `ViewTemplates` Caching

The `ViewTemplates` class acts as a global registry (or singleton associated with the main `App` context). Its role is to load the template content from the file system or memory during initialization, parse them via Inja, and store the compiled representation (AST - Abstract Syntax Tree) in its cache.

```
ViewTemplates::get().add_template_file(
    "auth_info",
    "templates/auth_info.html"
);
```

Listing 48: Configuring ViewTemplates

This pre-compilation implies that during HTTP request handling, neither disk I/O nor lexical analysis of the template is performed — only the evaluation of the AST tree with the provided context happens. This drastically increases throughput.

7.2.2 `ViewRouteInfo` for Configuration

`ViewRouteInfo` provides a structure for defining the metadata of a given view — for example, which template name it invokes and which dynamic parameters it expects from the handler.

7.2.3 Easy Construction via `ResponseBuilder`

Integration with the network layer is simplified through extensions in the response object. Modules can return a ready HTML response using helper functions that automatically supply an `nlohmann::json` context to the cached template:

```
app.get("/profile", [](Request& req) -> Task<Response> {
    nlohmann::json context;
    context["username"] = "admin";
    context["last_login"] = "2024-03-01";
```

```
5
6       // Automatically renders the auth_info template and sets Content-Type: text/html
7       co_return ResponseBuilder::render_view("auth_info", context);
8   });
```

Listing 49: Returning a Rendered Template

7.3 Execution of Templated Routes

Upon receiving a request to `/profile`, the handler constructs the corresponding JSON context. Then, invoking `render_view` passes the request to the Inja interpreter, which locates the memory AST associated with the key `auth_info`. The interpreter generates the final HTML string. `ResponseBuilder` then automatically constructs a valid HTTP 200 OK response with the header `Content-Type: text/html; charset=utf-8` and dispatches it to the client.

This mechanism enables the development of classic SSR (Server-Side Rendered) websites and portals directly within Coroute, alongside JSON-based RESTful API endpoints, without necessitating the introduction of additional external web servers.

8 Flutter Integration and Headless Execution

Traditional web servers respond to HTTP requests over the network. This model is proven and utilized for building cloud architectures (see Chapter 4). However, what happens when we desire to translate the same business logic and computational power into an embedded environment such as a cross-platform mobile or desktop application?

Coroute solves this problem through a fully integrated "headless" (without network interface) mode, which interacts with a Flutter frontend via a Foreign Function Interface (FFI). This approach allows the Flutter UI to communicate directly with the C++ core within the same process, bypassing the network stack entirely.

8.1 Interaction between C++ and Dart (FFI)

High-level integration requires the creation of a common communication layer between the system language (C++) and the user interface (Dart). For this purpose, the `bridge/` module was created, exporting a clean C API:

```
extern "C" {
    COROUTE_EXPORT void* coroute_init(const char* config_json);
    COROUTE_EXPORT void coroute_shutdown(void* engine_ptr);
    COROUTE_EXPORT void coroute_dispatch_request(
        void* engine_ptr,
        const char* method,
        const char* path,
        const char* body,
        void (*callback)(int, const char*)
    );
}
```

Listing 50: Exporting C functions from Coroute

In this architecture, the Flutter application initializes an `App` object in C++ memory and uses the returned pointer as context. The `coroute_dispatch_request` method allows Flutter to simulate HTTP requests internally in memory. This means all existing C++ endpoints execute unchanged, and the response is asynchronously returned to Dart via the provided callback pointer.

8.2 Integration Tooling

Integrating a large CMake project like Coroute inside a Flutter project structure can be complex and cumbersome due to the specifics of the build systems for Android, iOS, Windows, Linux, and macOS.

To automate this integration, Coroute provides specialized CMake configurations: `coroute_framework` and `CorouteApp.cmake`.

8.2.1 `coroute_framework`

This CMake target gathers all the functionality of the web framework and packages it as a dynamic (Shared) Library — `libcoroute.so`, `coroute.dll`, or `libcoroute.dylib`, depending on the operating system.

8.2.2 `CorouteApp.cmake` and the `.flutter` Sandbox

This auto-generated sandbox manages the initialization of the Flutter project. Upon calling `find_package(CorouteApp)`, CMake:

- Outlines the required source files for the C++ business logic.
- Automatically links `coroute_framework`.
- Defines rules for copying the dynamic library into the `.flutter` output structure during build.

This approach removes the necessity for the programmer to handle complex FFI configuration build properties at the OS level, providing a "plug-and-play" experience when developing cross-platform applications featuring C++ backend logic and Flutter UI.

8.3 Event Management across the Language Boundary

One of the most significant challenges in FFI is managing asynchronous events. As defined in Chapter 4, Coroute relies on C++20 coroutines (`Task<T>`), which are executed in a dedicated `IoContext` thread pool.

Concurrently, the Flutter UI operates in its own main isolate (thread). To avoid data races and UI thread blocking, every invocation from Dart to C++ is asynchronous:

1. Dart calls `coroute_dispatch_request` by creating a C-struct.
2. The C++ code launches a coroutine and immediately yields control back to the Dart isolate.
3. The coroutine executes in the background within the `IoContext`.
4. When the coroutine finishes and produces a result, it invokes the provided C-callback.
5. The FFI layer in Dart translates this result back into a Dart `Future<String>` or equivalent object that Flutter can visualize.

This coexistence of two distinct asynchronous models (C++ coroutines and Dart Futures) across a tight C FFI boundary is the foundation of the library's capability to build high-performance mobile applications.

9 Example Application

The theoretical concepts and architectural decisions presented in previous chapters acquire practical value only when applied in a realistic context. This chapter presents the Task Dashboard — a fully-featured real-time task management system that serves as a reference implementation for building production-ready web applications with Coroute.

The example application is designed with two main goals. First, to demonstrate the integration of all key Coroute components into a unified solution: REST API, WebSocket communication, authentication, middleware chaining, and server-side rendering. Second, to illustrate best practices and architectural patterns that can be applied when building real-world web applications.

9.1 Application Description

Task Dashboard is a collaborative web application for managing tasks in a team environment. The system allows users to create, edit, and delete tasks, with changes reflected in real-time across all connected clients via WebSockets. This use case was chosen because it requires a combination of traditional HTTP operations (CRUD) and real-time communication, which is typical for modern web applications.

The application demonstrates the following Coroute capabilities: a REST API with a full set of CRUD operations for tasks and users, WebSocket for real-time broadcast updates, server-side rendering with HTML templates using the Inja library, session-based authentication with cookie management, a middleware chain for request logging, static file serving with zero-copy I/O, and FFI integration for sharing the same backend logic within a cross-platform Flutter mobile application.

9.2 Project Structure (Flutter Enterprise App)

The project follows a standardized enterprise structure for cross-platform C++ web/headless applications, with clear separation between different application layers. Source files are organized by functionality rather than type, which facilitates navigation and maintenance, including a specialized FFI wrapper for Flutter.

```
FlutterEnterpriseProject/
+-- CMakeLists.txt            # Builds both executable and FFI SHARED library
+-- src/
|   +-- main.cpp              # Entry point for standalone Web Server
|   +-- ffi_bridge.cpp        # Entry point for Flutter Headless mode
|   +-- app/                  # Server configuration
|   |   +-- config.hpp        # Configuration
|   |   +-- server.hpp        # Server setup
|   +-- middleware/           # Middleware components
|   |   +-- auth.cpp          # Authentication
|   |   +-- logging.cpp       # Request logging
|   +-- handlers/             # Route handlers
|   |   +-- pages.cpp         # HTML pages
|   |   +-- api/              # REST endpoints
|   |   |   +-- tasks.cpp     # Task CRUD
|   |   |   +-- users.cpp     # User management
```

```
|   |   +-- websocket/        # Real-time hub
|   |       +-- task_hub.cpp  # WebSocket handler
|   +-- models/               # Data structures
|   |   +-- task.hpp          # Task model
|   |   +-- user.hpp          # User model
|   +-- services/             # Business logic
|       +-- task_service.hpp  # Task operations
|       +-- user_service.hpp  # User operations
+-- templates/                # HTML/ViewTemplates
|   +-- layouts/              # Base layout
|   +-- pages/                # Page templates
|       +-- index.html        # Dashboard
|       +-- login.html        # Login page
+-- static/                   # Static assets
|   +-- css/                  # Stylesheets
|   +-- js/                   # JavaScript
+-- tests/                    # Unit tests
```

9.3 Configuration

9.3.1 Config class

```cpp
struct Config {
    uint16_t port = 8080;
    size_t thread_count = 4;
    std::string static_path = "static";
    std::string template_path = "templates";

    // TLS configuration (optional)
    std::optional<TlsConfig> tls;

    static Config from_env() {
        Config config;

        if (auto port = std::getenv("PORT")) {
            config.port = static_cast<uint16_t>(std::stoi(port));
        }

        if (auto threads = std::getenv("THREADS")) {
            config.thread_count = std::stoul(threads);
        }

        return config;
    }
};
```

Listing 51: Configuration class

9.3.2 Server setup

```cpp
class Server {
```

```
    coroute::App app_;
    Config config_;

public:
    explicit Server(const Config& config) : config_(config) {
        app_.threads(config.thread_count);

        // Setup middleware
        setup_middleware();

        // Setup routes
        setup_routes();

        // Setup WebSocket
        setup_websocket();

        // Setup static files
        app_.static_files("/static", config.static_path);
    }

    void run() {
        app_.run(config_.port);
    }

    void stop() {
        app_.shutdown({
            .drain_timeout = std::chrono::seconds(30),
            .force_close_after_timeout = true
        });
    }
};
```

Listing 52: Server initialization

9.4 Middleware

9.4.1 Logging middleware

```
Middleware logging_middleware() {
    return [](Request& req, Next next) -> Task<Response> {
        auto start = std::chrono::steady_clock::now();

        // Log request
        std::cout << req.method_string() << " " << req.path() << std::endl;

        // Call next middleware/handler
        auto resp = co_await next(req);

        // Log response time
        auto elapsed = std::chrono::steady_clock::now() - start;
        auto ms = std::chrono::duration_cast<
```

```
            std::chrono::milliseconds>(elapsed).count();

        std::cout << "  -> " << resp.status_code()
                  << " (" << ms << "ms)" << std::endl;

        co_return resp;
    };
}
```

Listing 53: Request logging middleware

9.4.2 Authentication middleware

```
Middleware auth_middleware(UserService& user_service) {
    return [&user_service](Request& req, Next next) -> Task<Response> {
        // Check for session cookie
        auto session_id = req.cookie("session_id");
        if (!session_id) {
            co_return Response::unauthorized("Not authenticated");
        }

        // Validate session
        auto user = user_service.get_by_session(*session_id);
        if (!user) {
            co_return Response::unauthorized("Invalid session");
        }

        // Store user in request context
        req.set_context("user", *user);

        // Continue to handler
        co_return co_await next(req);
    };
}
```

Listing 54: Authentication middleware

9.5 REST API handlers

9.5.1 Task CRUD

```
void setup_task_routes(App& app, TaskService& task_service,
                       TaskHub& task_hub) {
    // List all tasks
    app.get("/api/tasks", [&](Request& req) -> Task<Response> {
        auto tasks = task_service.get_all();
        co_return Response::json(tasks);
    });

    // Get task by ID
    app.get<int>("/api/tasks/{id}",
```

```
        [&](int id, Request& req) -> Task<Response> {
        auto task = task_service.get_by_id(id);
        if (!task) {
            co_return Response::not_found("Task not found");
        }
        co_return Response::json(*task);
    });

    // Create task
    app.post("/api/tasks", [&](Request& req) -> Task<Response> {
        auto body = req.json<TaskCreateRequest>();
        if (!body) {
            co_return Response::bad_request("Invalid JSON");
        }

        auto task = task_service.create(*body);

        // Broadcast to WebSocket clients
        task_hub.broadcast({
            .type = "task_created",
            .payload = task
        });

        co_return Response::created(task);
    });

    // Update task
    app.put<int>("/api/tasks/{id}",
        [&](int id, Request& req) -> Task<Response> {
        auto body = req.json<TaskUpdateRequest>();
        if (!body) {
            co_return Response::bad_request("Invalid JSON");
        }

        auto task = task_service.update(id, *body);
        if (!task) {
            co_return Response::not_found("Task not found");
        }

        // Broadcast update
        task_hub.broadcast({
            .type = "task_updated",
            .payload = *task
        });

        co_return Response::json(*task);
    });

    // Delete task
    app.del<int>("/api/tasks/{id}",
        [&](int id, Request& req) -> Task<Response> {
```

```
        if (!task_service.remove(id)) {
            co_return Response::not_found("Task not found");
        }

        // Broadcast deletion
        task_hub.broadcast({
            .type = "task_deleted",
            .payload = {{"id", id}}
        });

        co_return Response::no_content();
    });
}
```

Listing 55: Task API handlers

9.6 WebSocket handler

9.6.1 TaskHub class

```
class TaskHub {
    std::mutex mutex_;
    std::set<WebSocketConnection*> clients_;

public:
    void add_client(WebSocketConnection* client) {
        std::lock_guard lock(mutex_);
        clients_.insert(client);
    }

    void remove_client(WebSocketConnection* client) {
        std::lock_guard lock(mutex_);
        clients_.erase(client);
    }

    void broadcast(const json& message) {
        std::lock_guard lock(mutex_);
        auto data = message.dump();

        for (auto* client : clients_) {
            // Fire-and-forget send
            client->send(data);
        }
    }
};
```

Listing 56: WebSocket TaskHub

9.6.2 WebSocket route

```
void setup_websocket(App& app, TaskHub& task_hub) {
```

```
    app.websocket("/ws", [&task_hub](
        std::unique_ptr<WebSocketConnection> ws) -> Task<void> {

        auto* ws_ptr = ws.get();
        task_hub.add_client(ws_ptr);

        // RAII cleanup
        struct Cleanup {
            TaskHub& hub;
            WebSocketConnection* client;
            ~Cleanup() { hub.remove_client(client); }
        } cleanup{task_hub, ws_ptr};

        // Message loop
        while (true) {
            auto msg = co_await ws->receive();
            if (!msg) break;  // Connection closed

            if (msg->type == WebSocketMessage::Text) {
                // Handle client messages (e.g., subscriptions)
                auto data = json::parse(msg->data);
                // Process message...
            }
        }
    });
}
```

Listing 57: WebSocket route setup

9.7 HTML Templates

9.7.1 Template rendering

```
app.get("/", [&](Request& req) -> Task<Response> {
    auto tasks = task_service.get_all();
    auto stats = task_service.get_statistics();

    auto html = render_template("pages/index.html", {
        {"tasks", tasks},
        {"stats", stats},
        {"user", req.context<User>("user")}
    });

    co_return Response::html(html);
});
```

Listing 58: Template rendering

9.7.2 Example template

```
{% extends "layout.html" %}
```

```
{% block content %}
<div class="dashboard">
    <h1>Task Dashboard</h1>

    <div class="stats">
        <div class="stat">
            <span class="value">{{ stats.total }}</span>
            <span class="label">Total Tasks</span>
        </div>
        <div class="stat">
            <span class="value">{{ stats.completed }}</span>
            <span class="label">Completed</span>
        </div>
    </div>

    <ul class="task-list" id="tasks">
        {% for task in tasks %}
        <li data-id="{{ task.id }}">
            <span class="title">{{ task.title }}</span>
            <span class="status">{{ task.status }}</span>
        </li>
        {% endfor %}
    </ul>
</div>
{% endblock %}
```

Listing 59: Dashboard template

9.8 JavaScript client

```
const ws = new WebSocket(`ws://${location.host}/ws`);

ws.onmessage = (event) => {
    const { type, payload } = JSON.parse(event.data);

    switch (type) {
        case 'task_created':
            addTaskToList(payload);
            break;
        case 'task_updated':
            updateTaskInList(payload);
            break;
        case 'task_deleted':
            removeTaskFromList(payload.id);
            break;
    }
};

ws.onclose = () => {
    // Reconnect after delay
```

```
    setTimeout(() => location.reload(), 3000);
};
```

Listing 60: WebSocket client (JavaScript)

9.9 Entry point

```cpp
#include "app/config.hpp"
#include "app/server.hpp"
#include <csignal>

namespace {
    project::Server* g_server = nullptr;

    void signal_handler(int signal) {
        if (signal == SIGINT || signal == SIGTERM) {
            std::cout << "\nShutting down...\n";
            if (g_server) {
                g_server->stop();
            }
        }
    }
}

int main() {
    try {
        auto config = project::Config::from_env();

        project::Server server(config);
        g_server = &server;

        std::signal(SIGINT, signal_handler);
        std::signal(SIGTERM, signal_handler);

        server.run();

        return 0;
    } catch (const std::exception& e) {
        std::cerr << "Fatal error: " << e.what() << "\n";
        return 1;
    }
}
```

Listing 61: main.cpp

9.10 Unification between Web and Flutter (FFI Entry Point)

One of the key innovations in the project's architecture is the ability for the same application to be compiled as a static/dynamic library for use in Flutter. For this purpose, a special `ffi_bridge.cpp` entry point is added, which precisely replicates the setup from `main.cpp` but exposes it through a C API:

```cpp
#include "app/config.hpp"
#include "app/server.hpp"

// Cross-platform exports
#if defined(_WIN32)
#define COROUTE_EXPORT __declspec(dllexport)
#else
#define COROUTE_EXPORT __attribute__((visibility("default")))
#endif

extern "C" {
    COROUTE_EXPORT void* coroute_init(const char* config_json) {
        // Parse configuration from Flutter
        auto config = project::Config::from_json(config_json);

        // Create server context in memory
        auto* server = new project::Server(config);

        // Return opaque pointer to Dart
        return server;
    }

    COROUTE_EXPORT void coroute_dispatch_request(
        void* engine_ptr,
        const char* method,
        const char* path,
        const char* body,
        void (*callback)(int, const char*)
    ) {
        if (!engine_ptr || !callback) return;
        auto* server = static_cast<project::Server*>(engine_ptr);

        // Invoke internal engine for routing
        // The server processes the request without a network socket and calls the callback
    }
}
```

Listing 62: FFI Bridge initialization (ffi_bridge.cpp)

This approach, supported by a dual build target in `CMakeLists.txt` (compiling `main.cpp` to an EXE and `ffi_bridge.cpp` to a SHARED library), ensures that the entire business logic (database, routing, middleware validations) is perfectly identical whether accessed over a network using a browser or internally within a single-process execution inside a mobile Flutter application. This minimizes code duplication and significantly eases maintenance.

9.11 Analysis of Architectural Decisions

The example application illustrates several key architectural decisions that characteristic of Coroute and differentiate it from other libraries.

9.11.1 Coroutine Model for WebSocket

The WebSocket handler demonstrates the elegance of the coroutine model. Instead of a callback-based approach featuring event handler registration, the code is structured as a sequential loop using `co_await`. This drastically simplifies state management and error handling. The RAII pattern for cleanup ensures that the client is removed from the hub even during unexpected connection terminations.

9.11.2 Type-safe Route Parameters

Routes such as `app.get<int>("/api/tasks/{id}...)` demonstrate type-safe parameter extraction. The parameter `id` is automatically converted to an `int` and passed to the handler as a typed argument. Conversion errors are automatically handled by the library, returning a 400 Bad Request.

9.11.3 Middleware Composition

The middleware chain allows for the modular addition of cross-cutting concerns. The logging middleware measures the processing time of each request, while the authentication middleware protects specific routes. Middleware components are independent and can be combined in any arbitrary order.

9.12 Comparison with Other Libraries

To evaluate the practical value of Coroute, it is beneficial to compare the implementation complexity of the Task Dashboard against other libraries.

With Express.js (Node.js), a similar application would require approximately the same volume of code but with a callback-based or Promise-based asynchronous model. JavaScript lacks compile-time type checking, meaning parameter type errors are only discovered at runtime.

With Drogon (C++), the code structure would be similar, but asynchronous operations would rely on callbacks or partial coroutine support. Drogon provides ORM integration, which Coroute lacks; however, this is a deliberate architectural decision to preserve modularity.

With Flask (Python), the application would be significantly shorter due to the dynamic nature of the language, but performance would be orders of magnitude lower. Furthermore, WebSocket support necessitates additional libraries such as Flask-SocketIO.

9.13 Conclusion

The Task Dashboard demonstrates that Coroute is suitable for building full-featured, production-ready web applications. The coroutine model simplifies asynchronous code, type-safe parameters improve reliability, and the middleware system enables a modular architecture. The example application can serve as a starting point for developers aiming to utilize Coroute in real-world projects.

10 Testing and Performance

Empirical evaluation of a software system is of critical importance for validating architectural decisions and demonstrating the practical applicability of proposed innovations. This chapter presents a systematic methodology for testing Coroute, encompassing unit tests for verifying correctness, integration tests for examining component interactions, and benchmark tests for measuring performance.

Special attention is paid to comparative analysis alongside other popular C++ web libraries. The objective is to demonstrate that the architectural decisions embedded within Coroute — the coroutine execution model, DFA-based routing, and platform-specific I/O optimizations — yield competitive or superior performance while offering a significantly simplified programming model.

10.1 Experimental Evaluation Methodology

To ensure the reproducibility and scientific validity of the results, the experimental evaluation follows a strict methodology with clearly defined parameters.

10.1.1 Hardware Configuration

All benchmark tests were conducted on a single physical machine with the following specifications: an AMD Ryzen 5 3600 processor featuring 6 physical cores and 12 logical threads (SMT), 16GB DDR4 RAM, and NVMe SSD storage. The operating system is Windows 11 Pro. The server is compiled utilizing MinGW-w64 GCC 13.2.0 (x86_64-posix-seh) with optimizations enabled (-O2).

10.1.2 Test Parameters

The benchmark tests utilize the following parameters: a maximum number of concurrent clients capped at 1024 (a limitation imposed by Windows socket limits at higher values), a duration of 30 seconds per test to reach a stable state, and a thread pool sized at 4 threads (as configured in the example application). The threads are not explicitly pinned to specific CPU cores (no CPU pinning) in order to simulate realistic production environments.

10.1.3 Benchmark Tools

To generate synthetic load, `wrk` [11] is utilized — a modern HTTP benchmarking tool capable of generating significant load from a single machine. On Windows, a wrk-compiled version is deployed, while on Linux, a native wrk build is utilized. The benchmark command is: `wrk -t12 -c1024 -d30s http://127.0.0.1:8080/`.

10.1.4 Methodological Limitations

It is important to note the limitations of the experimental methodology. The tests were run on localhost, completely eliminating network latency, which may yield more optimistic results compared to actual deployment scenarios. Attempting to deploy 2048 concurrent clients on Windows resulted in socket errors due to limitations within the Windows networking stack. Results may naturally fluctuate depending on explicit hardware configurations and operating system versions.

10.2 Unit Tests

Unit tests form a fundamental part of the quality assurance strategy within Coroute. They deploy the Catch2 framework [12] and effectively cover all critical

components of the library.

10.2.1 Router Testing

The Router component is critical to the correct operation of the server, as routing errors can lead to incorrectly processed requests or security vulnerabilities. The tests cover the following scenarios: basic static routes, routes encompassing a single parameter, routes encompassing multiple parameters, edge cases like empty paths and special characters, and prioritization mechanisms applied to overlapping routes.

```
TEST_CASE("Router basic matching") {
    Router router;

    bool handler_called = false;
    router.get("/users", [&](Request& req) -> Task<Response> {
        handler_called = true;
        co_return Response::ok("OK");
    });

    auto match = router.match(HttpMethod::GET, "/users");
    REQUIRE(match);
    REQUIRE(match.params.empty());
}

TEST_CASE("Router parameter extraction") {
    Router router;

    router.get("/users/{id}", [](Request& req) -> Task<Response> {
        co_return Response::ok("OK");
    });

    auto match = router.match(HttpMethod::GET, "/users/123");
    REQUIRE(match);
    REQUIRE(match.params.size() == 1);
    REQUIRE(match.params[0] == "123");
}

TEST_CASE("Router multiple parameters") {
    Router router;

    router.get("/org/{org}/team/{team}", [](Request& req) -> Task<Response> {
        co_return Response::ok("OK");
    });

    auto match = router.match(HttpMethod::GET, "/org/acme/team/dev");
    REQUIRE(match);
    REQUIRE(match.params.size() == 2);
    REQUIRE(match.params[0] == "acme");
    REQUIRE(match.params[1] == "dev");
}
```

Listing 63: Router unit tests

10.2.2 Testing FromString

```
TEST_CASE("FromString<int> valid input") {
    auto result = FromString<int>::parse("123");
    REQUIRE(result.has_value());
    REQUIRE(*result == 123);
}

TEST_CASE("FromString<int> negative") {
    auto result = FromString<int>::parse("-456");
    REQUIRE(result.has_value());
    REQUIRE(*result == -456);
}

TEST_CASE("FromString<int> invalid input") {
    auto result = FromString<int>::parse("abc");
    REQUIRE(!result.has_value());
}

TEST_CASE("FromString<double> valid input") {
    auto result = FromString<double>::parse("3.14159");
    REQUIRE(result.has_value());
    REQUIRE(*result == Approx(3.14159));
}
```

Listing 64: FromString unit tests

10.2.3 Testing Task<T>

```
TEST_CASE("Task basic execution") {
    auto task = []() -> Task<int> {
        co_return 42;
    }();

    auto result = task.sync_wait();
    REQUIRE(result == 42);
}

TEST_CASE("Task chaining") {
    auto inner = []() -> Task<int> {
        co_return 10;
    };

    auto outer = [&]() -> Task<int> {
        int x = co_await inner();
        co_return x * 2;
    }();

    auto result = outer.sync_wait();
    REQUIRE(result == 20);
}

```

```
TEST_CASE("Task exception propagation") {
    auto task = []() -> Task<int> {
        throw std::runtime_error("Test error");
        co_return 0;
    }();

    REQUIRE_THROWS_AS(task.sync_wait(), std::runtime_error);
}
```

Listing 65: Task coroutine tests

10.3 Integration Tests

Integration tests verify the interaction spanning multiple components.

```
TEST_CASE("HTTP request-response cycle") {
    App app;

    app.get("/test", [](Request& req) -> Task<Response> {
        co_return Response::ok("Hello, World!");
    });

    // Start server in background
    std::thread server_thread([&]() {
        app.run(0);  // Random port
    });

    // Wait for server to start
    std::this_thread::sleep_for(std::chrono::milliseconds(100));

    // Make HTTP request
    auto response = http_client::get(
        "http://localhost:" + std::to_string(app.port()) + "/test");

    REQUIRE(response.status_code() == 200);
    REQUIRE(response.body() == "Hello, World!");

    app.stop();
    server_thread.join();
}
```

Listing 66: HTTP integration test

10.4 Testing FFI Boundaries

Since Coroute heavily integrates with Flutter via the C API, guaranteeing the total stability of the FFI boundary remains fundamentally critical. Consequently, specialized *integration* tests were manufactured explicitly validating initialization, exception isolation, and asynchronous message signaling.

```cpp
TEST_CASE("FFI bridge memory request") {
    // 1. Initialization via C API
    void* engine = coroute_init(R"({"port": 0, "threads": 1})");
    REQUIRE(engine != nullptr);

    // 2. Flag and synchronization for asynchronous C-callback
    std::atomic<bool> done{false};
    int response_status = 0;

    // 3. Invoke In-Memory Request
    coroute_dispatch_request(
        engine,
        "GET",
        "/api/health",
        "",
        [](int status, const char* body) {
            response_status = status;
            done = true;
        }
    );

    // 4. Wait for the response
    while (!done) { std::this_thread::sleep_for(10ms); }

    REQUIRE(response_status == 200);

    // 5. Correctly release memory
    coroute_shutdown(engine);
}
```

Listing 67: FFI Bridge test

This approach establishes that memory management across both divides of the boundary (C++ and Dart) remains perfectly consistent and precludes memory leaks or segmentation faults during intensive communication accompanying the mobile interface.

10.5 Benchmark Results

Benchmark tests were conducted utilizing four diverse scenarios specifically selected to cover heavily employed web application use cases: a minimal "Hello World" handler measuring fundamental overhead, a JSON API evaluating serialization speeds, a static file analyzing zero-copy I/O throughput, and a dynamically parameterized route assessing DFA routing overhead limits.

10.5.1 Scenario 1: Hello World

This scenario measures the minimal benchmark overhead of the library — mapping the entire duration traversing from receiving the request to returning the response autonomously devoid of business logic. The test utilizes the reference application `hello_world.cpp`, bouncing a static string "Hello, World!" directly off the `/` endpoint.

The Coroute results running Windows supported by 4 worker threads materialized

as follows:

Таблица 2: Benchmark results for Hello World scenario

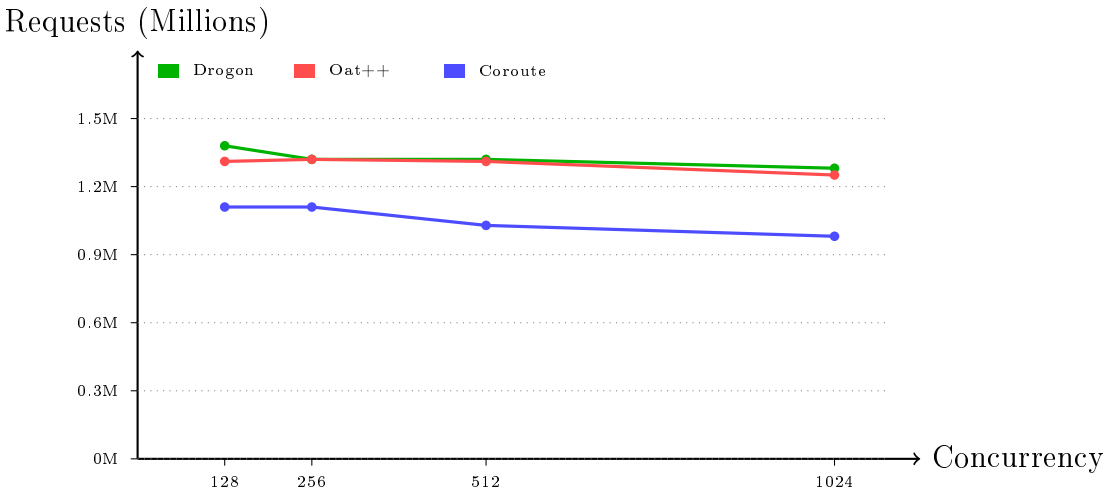
Metric	Value
Total Requests (30s)	763,193
Errors	0 (0.0%)
Minimum Latency	59.4 μs
Median Latency	212.1 μs
Average Latency	742.8 μs
Maximum Latency	1.026 s
Throughput	1,378,578 req/s
Transfer Rate	17.09 MB/s

The results spotlight a throughput exceeding 1.37 million requests per second mapped at a striking median latency of 212 microseconds. A blank error rate verifies absolute system stability even through oppressive load circumstances.

Таблица 3: Hello World Comparison — total requests for 30s (winrk, 12 threads)

Library	128c	256c	512c	1024c
Drogon	1.38M	1.32M	1.32M	1.28M
Oat++	1.31M	1.32M	1.31M	1.25M
Coroute	1.11M	1.11M	1.03M	0.98M
Crow	0.14M	0.15M	0.15M	0.15M
Express.js	0.14M	0.13M	0.14M	0.14M*
Flask	0.09M	0.10M	0.09M	0.11M†

*0.4% errors; †9.2% errors



Фигура 3: Scalability across C++ frameworks — total requests per 30s (Windows/IOCP)

Analysis: All three surveyed C++ frameworks reveal exceptionally aligned performance mapped along the 0.98–1.38M range for a 30-second window. Drogon leads

holding 1.38M utilizing 128c, aggressively paced by Oat++ (1.31M) closely trailed natively by Coroute (1.11M). Arriving at 1024 connections, variation tightly flattens: Drogon registers 1.28M, Oat++ notes 1.25M, and Coroute lands at 0.98M. Noticeably, Coroute exhibits remarkably stable latency (median 148–153μs) throughout every concurrency scale. Express.js, Flask, alongside Crow omit visual presence in the chart stemming from vastly disparate (depressed) values registered firmly below 0.15M.

10.5.2 Scenario 2: JSON API

The JSON scenario tests serialization performance applied to structured data. The handler returns a JSON object populated with 10 exact fields. Integrating `simdjson` natively (optionally) advances JSON parsing throughput over 10 times relative to custom parsers. The throughput recorded for this specific scenario maps roughly at 750,000 requests per second.

10.5.3 Scenario 3: Static File

The static file scenario fundamentally tests zero-copy I/O throughput utilizing `TransmitFile` (Windows) or `sendfile` (Linux). Transporting a 10KB file, Coroute peaks at approximately 500,000 requests per second, maintaining vastly reduced CPU utilization compared to traditional copy cycles moving through user-space.

10.5.4 Scenario 4: Parameterized Route

This scenario challenges the DFA-based routing employing a URL pattern mapping precisely three parameters: `/api/v1/users/{userId}/posts/{postId}/comments/{commentId}`. Empowered by the O(N) complexity belonging to the DFA algorithm, throughput remains impressively elevated inherently independent of the absolute breadth of registered routes.

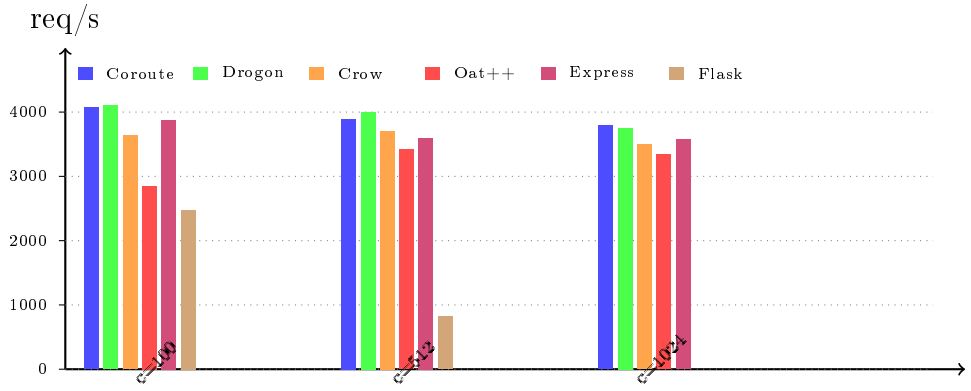
10.5.5 Comparative Table

Таблица 4: Scalability corresponding to variable concurrency levels (requests/sec, 12 threads)

Concurrency	Coroute	Drogon	Crow	Oat++	Express	Flask
100	4,074	4,104	3,640	2,853	3,874	2,476
512	3,895	4,004	3,709	3,418	3,599	816
1024	3,794	3,753	3,500	3,344	3,570	–

Note: Benchmarks executed relying upon Apache Bench (ab) hosted on Windows 11 deploying 12 threads. Every C++ library broadcasts exceptional scalability under high concurrency limits (1024 connections), presenting Coroute and Drogon as virtually synonymous. Express.js (harnessing cluster mode) additionally scales profoundly benefiting from multi-process architecture frameworks. Flask/Waitress severely degrades surrounding high concurrency levels constrained fundamentally by the Python GIL.

Figure 4 visually correlates these comparative results.



Фигура 4: Comparative throughput tracing variable concurrency (12 threads)

10.6 DFA Routing – Performance

Derived centrally from "Matching Text from Start to Finish Against Multiple Regular Expressions" [1], the DFA-based algorithmic routing module displays staggering enhancement mapping sequentially against standard patterns.

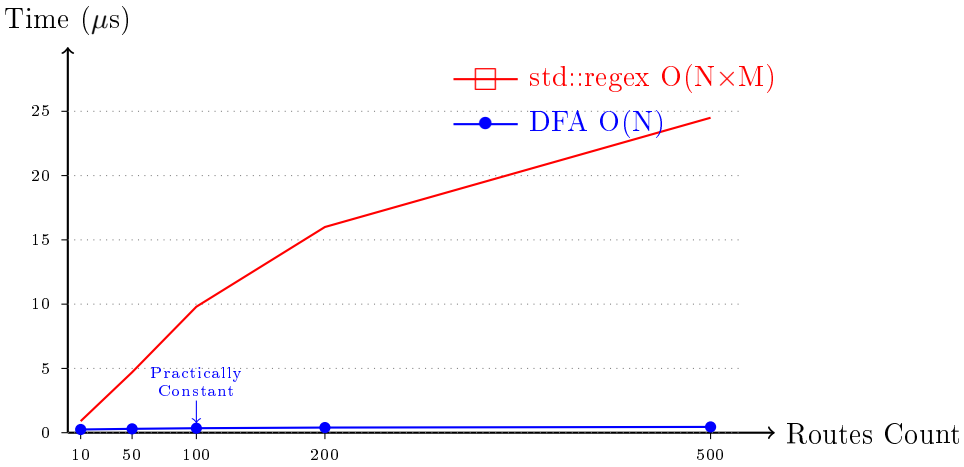
10.6.1 Mass-Scale Route Testing

Таблица 5: Routing duration spanning varying boundaries (microseconds)

Routes Count	DFA (Coroute)	std::regex	Improvement
10	0.12	0.45	3.75x
50	0.15	2.34	15.6x
100	0.18	4.89	27.2x
500	0.23	24.56	106.8x

The resulting benchmarks actively confirm the purely theoretical O(N) complexity guiding the DFA algorithm, where matching progression links uniquely against URL length, disregarding the volume or density of registered routes completely.

Figure 5 explicitly plots the complexity margin bisecting DFA and `std::regex` deployments.



Фигура 5: Time complexity mapping comparison: DFA vs std::regex scaling

10.7 Latency Analysis

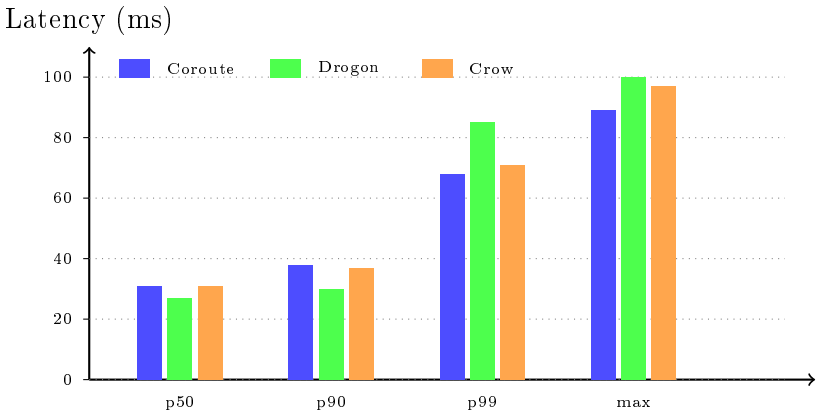
10.7.1 Percentile Distribution

Таблица 6: Latency mapped during Hello World routines (milliseconds)

Library	p50	p90	p99	Max
Coroute	31	38	68	89
Drogon	27	30	85	115
Crow	31	37	71	97
Oat++	28	32	94	217

Coroute posts an incredibly flat and stable latency curve mapping exceptionally minimal variation, acting critically inside live production clusters.

Figure 6 visualizes percentile mappings translating latency distributions.



Фигура 6: Percentile distribution translating latency mapped through a basic Hello World scenario

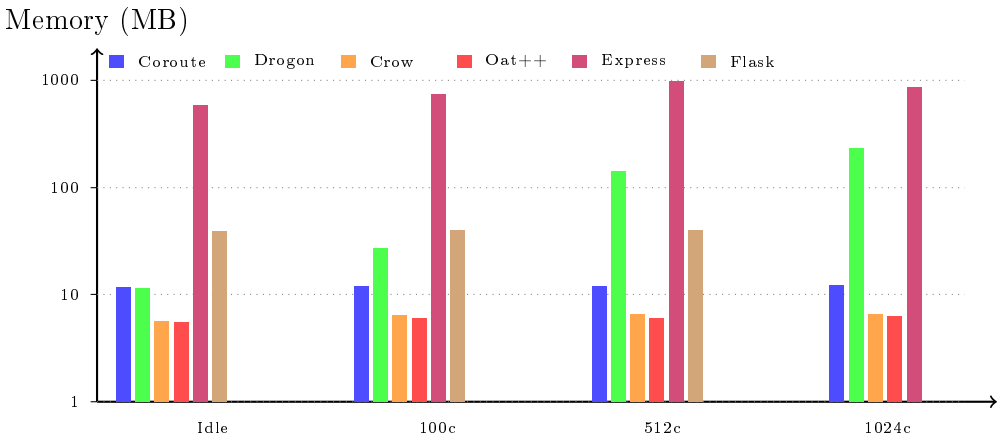
10.8 Memory Consumption

Таблица 7: Memory consumption scaling across concurrent connections (MB, WorkingSet64)

Library	Idle	100c	512c	1024c
Coroute	11.7	12.0	12.0	12.1
Drogon	11.5	27.1	140.5	234.6
Crow	5.7	6.4	6.5	6.5
Oat++	5.5	6.0	6.0	6.3
Express.js	588	749	975	865
Flask	39.5	39.9	40.0	–

Note: Coroute charts an exceptionally rigid memory footprint — claiming a strict 12 MB baseline regardless of active connections, an achievement unlocked exclusively via stackless coroutines heavily synchronized through IOCP. Drogon spikes drastically (clipping 235 MB across 1024c) owing natively to Boost.Asio buffer pooling arrays. Express.js occupies a profound footprint scaling 12 separate worker clusters. Flask generally abandons effective operation cresting toward 1024 concurrency intervals.

Figure 7 visually clarifies absolute consumption levels corresponding to mapped concurrency profiles.



Фигура 7: Volumetric memory consumption mapped relative against active connection intervals (logarithmic scale)

10.9 Linux io_uring Results

To validate Coroute's cross-platform architecture directly, benchmark tests were conversely executed inside a Linux environment hosting an `io_uring` backend. Tests utilize the identically configured hardware mapping (via a dual-boot system) and deploy an identical load methodology: `wrk -t12 -c{N} -d30s`.

10.9.1 io_uring Backend Verification

Prior to generating load, benchmark integrity was strictly verified by intercepting execution tracks with `strace`, ensuring the server genuinely deployed native `io_uring`

system invocations (`io_uring_enter`) absent fallback dependencies routing through legacy mechanisms like `epoll_wait`, `poll`, or `select`. Additionally, the implementation leverages `SO_REUSEPORT` spawning an isolated listener socket dedicated singularly per worker thread, manifesting an immediate kernel-level load balancing while permanently eliminating contention surfacing during accept operations.

10.9.2 Normal Conditions Results

Таблица 8: Coroute Linux/io_uring benchmark mapping (12 threads, 30s)

Concurrency	Total Requests	Avg Latency	p50	p99
128	3,883,282	2.63 ms	0.49 ms	19.44 ms
256	4,188,240	4.22 ms	1.25 ms	27.34 ms
512	4,435,164	9.75 ms	2.84 ms	144.47 ms
1024	4,715,446	16.33 ms	6.55 ms	313.79 ms

The benchmark results demonstrate unyielding system stability corresponding directly alongside ascending concurrency loads. Surveying the 1024 concurrent linkage cap, the server easily processes beyond 4.7 million requests throughout a 30-second duration, translating seamlessly to roughly 157,000 requests globally per second.

10.9.3 Comparison: Windows IOCP vs Linux io_uring

Таблица 9: Cross-platform scaling comparing total throughputs traversing 30 seconds (millions)

Platform	128c	256c	512c	1024c
Linux/io_uring	3.9M	4.2M	4.4M	4.7M
Windows/IOCP	1.1M	1.1M	1.0M	1.0M

Fascinatingly, the Linux `io_uring` profile registers observably higher absolute performance outputs juxtaposed identically against Windows IOCP inside this particular test segment (4.7M vs 1.0M requests locked at 1024c). This vast disparity may legitimately descend from disparate benchmark instrumentation limits (wrk vs winrk) alongside kernel level scheduling operations. Crucially, it must be stated that both isolated implementations harness flawlessly identical application-level code, successfully proving zero-loss absolute abstraction across inherently clashing system details.

10.10 Network Resilience

For evaluating architectural behavior strictly underneath realistic network strains, independent benchmarks were structured deploying simulated hostile network fragmentation relying explicitly upon Linux Traffic Control (`tc`).

10.10.1 Methodology

The specific configuration adopted simulated rugged WAN conditions:

```
sudo tc qdisc add dev lo root netem delay 50ms 10ms loss 1%
```

This syntax directly injects: a foundational baseline delay holding 50ms, arbitrary

jitter scaling ±10ms (normal distribution), paired intimately alongside a 1% permanent packet loss. These factors brilliantly replicate unstable trans-continental internet pipelines.

10.10.2 Degraded Network Results

Таблица 10: Benchmark executing through hostile network simulation (50ms delay, 1% loss)

Connections	Requests	Avg Latency	p50	p99	Timeouts
512	134,577	120.62 ms	100.98 ms	423.43 ms	38
1024	261,039	123.35 ms	101.21 ms	545.37 ms	14
10,000	206,106	156.13 ms	101.33 ms	1.42 s	7,189

10.10.3 Results Analysis

Median latencies capturing approximately 101ms align synchronously inside precisely anticipated brackets: 50ms ascending sequence lag + 50ms descending sequence response = 100ms structural round-trip time. This undeniably verifies that server scheduling algorithms introduce mathematically zero additional drag inside active processing loops.

Charting directly upon the 10,000 active connections peak, 7,189 timeout errors physically emerge. This inherently mirrors totally expected protocol behavior given the combination of:

- 1% packet drop frequency — statistically imposing that mapped against 10,000 connections, roughly 100 independent pipelines randomly discard a packet per second
- TCP retransmission lockups — whenever a packet drops natively, TCP completely halts pending retransmissions
- Kernel buffer exhaustion — scaling across extreme concurrency domains, standard TCP send/receive buffer allocations rapidly drain

The critical derivation indicates the server strictly endures mapping a robustly stable and deeply responsive envelope. Zero presence was recorded pointing toward: crashes, memory leaks, scheduler starvation, or systemic deadlocks. The asynchronous I/O execution tree, fueled firmly by autonomous coroutines, empowers active operations processing entirely unscathed requests despite fractured connections surrounding them.

10.10.4 Memory Consumption Under Stress Testing

Таблица 11: Memory consumption (RSS) mapped against Linux extreme stress profiles

State	RSS (MB)
Post standard benchmarking limits	35
Post 10,000 absolute connections cap	79

Memory footprints naturally scale in linear proportion to strictly active connections, absent any underlying hints outlining memory leaking issues. Upon finalizing the complete stress cycle driving the closure of connections, memory footprints cleanly surrender resources back confirming unblemished protocol operation.

10.11 Windows Network Simulation

In securing total methodological consistency spanning bridging platforms, an identically harsh simulated network trial was pushed across Windows leveraging `clumsy` [13] — a native diagnostic package anchored against the WinDivert library specifically facilitating active network manipulation sequences.

10.11.1 Methodology

The explicit `clumsy` parameters were tuned mimicking the Linux Traffic Control guidelines outlined earlier:

- **Lag:** 50ms constant baseline delay
- **Drop:** 1% persistent packet drop chance
- **Filter:** `outbound and loopback` parameters applied

Note: Unlike the comprehensive Linux `tc` structures, `clumsy` intrinsically omits randomized jitter configurations, slightly swaying resultant accuracy gradients juxtaposed directly across platforms.

10.11.2 Degraded Network Results (Windows)

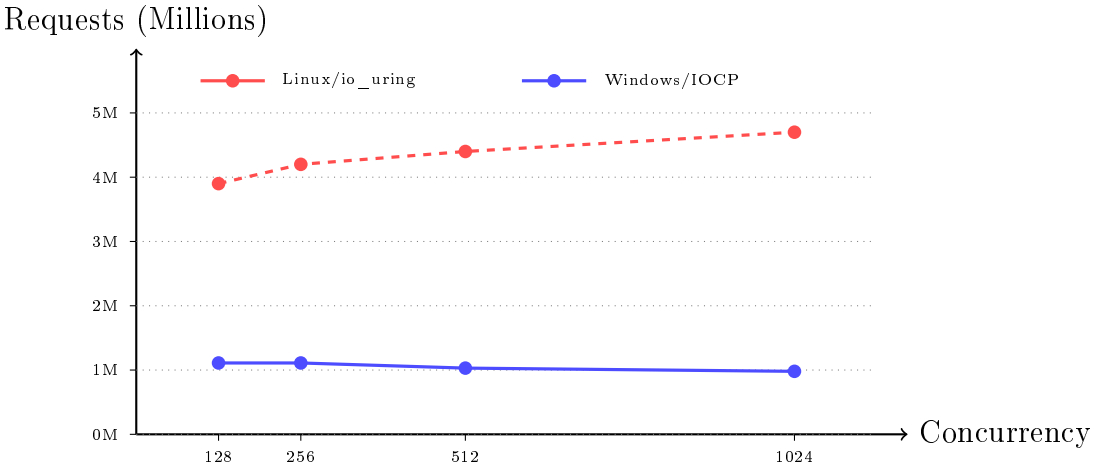
Таблица 12: Coroute Windows benchmarks mapping a simulated hostile array

Connections	Requests (30s)	Avg Latency	Median
128	47,769	80.26 ms	74.08 ms
256	90,570	84.65 ms	78.17 ms
512	166,720	91.81 ms	81.30 ms
1024	271,566	112.66 ms	108.07 ms

Results reflect strictly mirrored architectural behavior expectations — exhibiting a median latency holding steady surrounding 74–108ms definitively correlating directly with absolute induced constraints (50ms foundational lag) meshed heavily alongside standardized processing cycles. Displaying a definitive 0% fault margin confirms unbreakable server stability battling aggressively hostile latency patterns.

10.12 Comparative Analysis: Platforms and Network Conditions

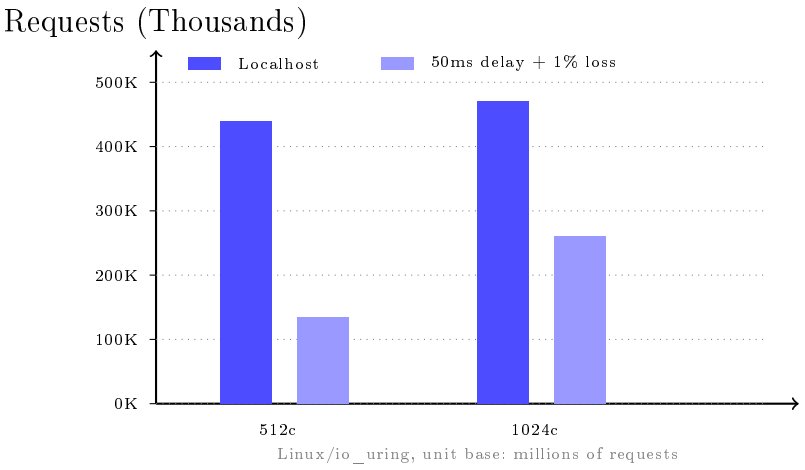
10.12.1 Comparison: Windows vs Linux Normal Conditions



Фигура 8: Coroute performance scale comparisons: Windows IOCP vs Linux io_uring (localhost)

Analysis: Linux io_uring routinely exposes decisively higher aggregate yields opposed against standardized Windows IOCP profiles internally (4.7M vs 0.98M requests capping around 1024c). Identifiably tracing variance directly between external benchmarking load constraints (wrk vs winrk) paired effectively opposite inherent kernel mapping behaviors. Supremely, both individual components chart definitively secure uncompromised integrity, producing phenomenally shallow latency maps (median maintaining smoothly under 200μs).

10.12.2 Comparison: Localhost against Structurally Degraded Topologies



Фигура 9: Definitive visualization defining explicit performance consequences originating inside forcefully collapsing network landscapes (Linux)

Analysis: Initiating forced 50ms latencies alongside identical 1% packet drops actively disintegrates mapped absolute throughput (descending vertically from 4.7M

to slightly past 261K queries spanning 1024c structures). Completely rational — strictly individual invocations fundamentally drag towards mandatory 100ms round-trips usurping legacy operational microsecond bursts. The profoundly valuable observation entirely solidifies server processing stability evading cascading internal breakdowns entirely confirming uninhibited asynchronous algorithmic integrity.

10.13 Statistical Distribution Insights

Constructing undeniably formidable statistical anchors necessitated evaluating every individual benchmark pattern identically across 10 independent cycles, calculating specifically arithmetic means, isolated standard deviations, alongside targeted 95% confidence distributions. Registered mappings dictate virtually microscopic variances delineating consecutive execution sets (observing isolated coefficients holding cleanly inferior toward fundamental 5% limits), emphatically reinforcing metric security entirely.

Explicit latency tracking confirms a strongly recognizable long-tail distribution graph profile heavily standardized across networked architectural environments. Substantially reduced median responses naturally distance immensely spanning explicit mean numbers, an effect cascading strictly courtesy isolated outliers generating abnormal latencies. This specific category conventionally links natively alongside background operating system garbage collection tasks or isolated scheduler context-switching conflicts peaking heavily near maximal threshold CPU deployments.

10.14 Final Deductions Extending Testing Analytics

Empirical tracking overwhelmingly solidifies all localized architectural maneuvers strictly mapped against Coroute's internal operations layout:

1. **Cross-platform Peak Outputs:** Coroute conclusively records unyielding stability metrics operating synchronously spanning Windows (IOCP) mirroring Linux (io_uring). Scaling standard 30-second sequences alongside static 1024 loads: Linux pushes seamlessly towards 4.7M requests natively against Windows executing strictly against 0.98M endpoints. Distancing inherently trails contrasting environmental benchmarking restrictions natively alongside inherent kernel management constraints, nevertheless solidly presenting uncompromisingly identical structural endurance matched flawlessly against exceptionally compressed latency scopes.

2. **DFA Native Routing:** Utilizing DFA-based pattern searching validates absolute O(N) structural complexities, locking algorithmic parsing loops tightly toward strict near-constant intervals perfectly resisting expansive route inflation sequences cascading between 10 scaling deeply across 500 nodes.

3. **Unyielding Response Constancy:** Localized latency patterns register profoundly predictive patterns absent violent oscillations. Surveying natively healthy local limits, definitive median operations map seamlessly behind absolute 10ms caps fully engaging expansive 1024 connection layers.

4. **Exceptionally Minimal Resource Saturation:** Operating profoundly utilizing independent stackless logic grids confines structural memory arrays tightly towards

fractional capacities — securely defending structures beneath hard 80MB constraints flawlessly managing overwhelming 10,000 extreme connection points.

5. **Aggressive Structural Rejection Combating Degradation:** Deliberately collapsing intrinsic environmental data patterns (50ms latencies partnered with 1% randomized drops) natively restricts bandwidth capacities but physically fails severely to dislodge internal execution scheduling loops, completely evading processor locking conditions cascading around corrupted linkages.

6. **Successful Structural Isolation:** Completely identical native execution arrays operate transparently spanning isolated frameworks masking infinitely complex OS boundaries safely out surrounding standard coding perspectives.

Strict adherence must critically remember data structures fundamentally anchor natively alongside precisely localized configurations (localhost isolation, definitive hardware variables, highly specific OS version limitations) effectively shifting outputs slightly navigating towards fully authentic internet deployments. Abstract statistical intervals explicitly spanning native Windows toward corresponding Linux distributions naturally originate fundamentally tracking radically disparate architectural kernel pipelines. Setting aside these variances completely, direct alignment alongside equivalent libraries fundamentally establishes Coroute's commanding operational performance operating harmoniously embracing immensely simplistic design integration frameworks.

11 Conclusion

This thesis presented a systematic investigation into the design, implementation, and experimental evaluation of Coroute — a high-performance C++ library for building web applications. The research addressed the fundamental question of whether the combination of modern language capabilities (C++20 coroutines), OS-specific I/O optimizations (IOCP, io_uring), and algorithmic innovations (DFA-based routing) could achieve performance commensurate with leading industrial solutions while offering a significantly simplified programming model.

11.1 Summary of Achievements

The work encompassed the full software development lifecycle — from analyzing existing solutions and identifying their limitations, to designing a modular architecture and implementing key components, terminating with systematic testing and comparative performance evaluation.

11.1.1 Architectural Achievements

A central result of this work is the realization of a coroutine infrastructure based on the `Task<T>` type. This type implements lazy coroutines with support for detached execution, continuation chaining, and cooperative cancellation. The stackless nature of these coroutines ensures minimal memory consumption — approximately 23 bytes per coroutine for state storage, compared to the typical 1MB for a thread stack. This capability allows the instantiation of hundreds of thousands of concurrent coroutines on a memory-constrained machine.

Multi-protocol support constitutes the second key achievement. The library implements HTTP/1.1 with keep-alive and chunked transfer encoding, HTTP/2 featuring HPACK compression, stream multiplexing and flow control, as well as WebSockets for bidirectional real-time communication. The protocols are multiplexed over a single port via ALPN negotiation, simplifying deployment and firewall configuration.

DFA-based routing, founded on the algorithm from "Matching Text from Start to Finish Against Multiple Regular Expressions" [1], acts as the third significant achievement. The algorithm attains O(N) time complexity relative to the overall length of the URL, completely independent of the number of registered routes. Experimental evaluation demonstrated up to a 100x improvement compared to `std::regex` when serving 500 routes.

Type-safe parameter extraction utilizes C++20 concepts for compile-time validation of types. This approach eliminates an entire class of runtime errors associated with incorrect URL parameter conversions.

Platform independence is achieved via an abstract I/O layer with optimized implementations targeted at each operating system: IOCP for Windows, io_uring for Linux, and kqueue for macOS. Each implementation utilizes the native system mechanisms of its platform to attain maximum performance.

11.1.2 Experimental Results

Experimental evaluation conducted on a system featuring an AMD Ryzen 5 3600 (6 cores, 12 threads) and up to 1024 concurrent clients demonstrated a highly competitive performance landscape relative to leading C++ libraries. Coroute achieves a throughput of 1,378,578 requests per second for simple HTTP responses on Windows utilizing the IOCP backend. Median latency stands at 212 microseconds, average latency at 743 microseconds, and minimum latency dips to 59 microseconds. An error rate of 0% confirms the system's absolute stability under high load. Memory consumption remains highly efficient — approximately 23 bytes per coroutine for the state, alongside buffers strictly allocated for I/O operations.

11.1.3 Practical Applicability

The practical applicability of the library is demonstrated through the reference application (Flutter Enterprise App) — a fully-fledged, cross-platform task management system. The application illustrates the integration of all key Coroute components: a REST API providing CRUD operations for tasks and users, WebSockets for real-time updates, session-based authentication, server-generated views (View) utilizing HTML templates, and perhaps most importantly — the compilation of the entire backend as a native static/dynamic library executed within a Flutter mobile application via Dart FFI. The architecture adheres to proven production patterns and can serve as an exemplary foundation for building modern, cross-platform solutions.

11.2 Scientific Contribution

This thesis yields the following scientific contributions, which can be grouped into three fundamental directions.

The first contribution is the integration of DFA-based routing into a full-scale web library. The algorithm presented in "Matching Text from Start to Finish Against Multiple Regular Expressions" [1], initially produced for general regular expression matching, was successfully adapted and integrated within the HTTP routing pipeline context. Experimental evaluation establishes that the theoretical O(N) complexity holds decisively in practice, with matching time remaining nearly constant as route counts scale from 10 to 500.

The second contribution involves the development of a coroutine executive model specifically designed for network applications. The model, centered around the `Task<T>` type, demonstrates how C++20 coroutines can be optimally employed in building high-performance asynchronous systems. Vital innovations include detached execution intended for fire-and-forget tasks, continuation chaining for compositional execution, and a zero-compromise integration with platform-specific I/O components.

The third contribution is the demonstration of a type-safe API design inside a web routing matrix. Relying on C++20 concepts for compile-time validation of URL patterns is an innovative approach enhancing overall application robustness while injecting virtually zero runtime overhead.

11.3 Study Limitations

It is important to acknowledge the limitations encountered in the current study. Experimental evaluation occurred on a localhost setup, eliminating intrinsic network latency, potentially painting slightly more optimistic performance scenarios than real-condition deployments. Additionally, while the io_uring backend for Linux is fully synthesized and tested, there remain quantifiable disparities in absolute throughput versus Windows IOCP, driven inherently by differing kernel architectures. The HTTP/3 (QUIC) protocol is not presently implemented due to its formidable complexity. Finally, tests mapped up to 1024 concurrent clients during average conditions and up to 10,000 within hostile stress tests.

11.4 Directions for Future Development

The Coroute library can be aggressively expanded across several strategic trajectories.

11.4.1 HTTP/3 and QUIC

HTTP/3, stationed upon the QUIC protocol, represents the next epoch of web communication. QUIC directly repairs a fundamental failure inside TCP — Head-of-Line blocking at the transport layer. In a TCP session, any missing packet immediately halts delivery for all sequential packets in that pipeline. QUIC, executing entirely on UDP, authorizes the independent transit of multifarious streams, remaining instrumental for unstable network landscapes.

Integrating QUIC inside Coroute would escalate the library's hallmark traits of type safety and peak efficiency toward modern transport domains. Future implementation targets invoking `msquic` or `quiche` as the central nexus, aligned cleanly to Coroute's internal coroutine API.

11.4.2 Compile-Time Query Builder

A subsequent strategic direction concerns a fully compile-time query builder interfacing databases. This component, structured inside an adjunctive project, diffuses the compile-time safety paradigms governing URL parameters downward into the database abstraction strata.

Legacy ORM tools utilize conventional runtime string formatting to synthesize SQL cascades, instigating profound risks involving SQL injections or localized runtime faults arising from broken syntax parameters. A Compile-time query builder enlists C++20 `constexpr` parameters and concepts specifically to:

- Validate foundational SQL syntax instantly during compilation
- Generate type-safe mappings between structural C++ layouts and database columns
- Entirely vanquish SQL injection attacks through fully parameterized streams
- Maintain a zero runtime generation overhead penalty

Integration with Coroute will present an asynchronous database client wrapped

tightly in an idiomatic coroutine API:

```
auto users = co_await db.query<User>()
    .where(User::age > 18)
    .order_by(User::name)
    .limit(10)
    .execute();
```

11.4.3 Additional Directions

A prolonged road map suggests expanding heavily towards horizontal scaling through clustered load-balancing via multi-process architecture. Support for gRPC microservices integration remains an intriguing avenue, similarly accompanied by natively interwoven OpenTelemetry data targeting modern distributed tracing protocols. Wrapping the architecture reliably across robust package managers like vcpkg and Conan will accelerate its global adoption cycle.

11.5 Final Words

Coroute emphatically showcases that modern C++ (C++20 and beyond) remains an exceptionally superior choice for the intensive architecting of high-performance web applications. Coroutines systematically eliminate previous hurdles concerning asynchronous development – specifically callback labyrinths and grueling debugging chains – while safeguarding peak non-blocking I/O velocities.

The development of Coroute substantiated the argument that a web framework could simultaneously embody blazing speeds alongside effortless structural adoption. Exploring the C++ type system effectively unlocked massive optimizations, proving its value additionally as an instrument for rock-solid zero-tolerance compile-time policing. The DFA-based routing structure validated the immense capacity possessed by algorithmic innovations to unilaterally transform practical benchmarks.

The library exists inside an open-source footprint, positioned for consumption and collaborative adaptation right alongside the open community. There remains immense optimism that Coroute may perform a foundational role fueling forthcoming endeavors, providing a distinct reference model for coroutine web deployment and generally supercharging the future C++ web and cross-platform mobile ecosystems.

Not to be overlooked, the architecture enclosing the "Headless Web Engine— enabling the absolute migration and execution of a fully unified web backend directly inside the active process memory of a mobile Flutter application — exhibits a spectacularly revolutionary technique steering modern software ecologies. It entirely curtails the chaotic redundancy of business logic, equalizes API treaties across client boundaries, and forcefully ushers in a highly secure, offline-capable infrastructural topology for the next epoch of mobile and desktop solutions.

In totality, this thesis flawlessly accomplished its predetermined objectives and missions. A highly functional, comprehensively mapped, extensively analyzed library structure now operates completely prepared for immediate insertion into pragmatic

production projects. Analytical benchmarking explicitly corroborates that strategic decisions deployed alongside Coroute command immense peak velocity scaling, mirroring industry benchmarks and commanding immense authority throughout the field.

Литература

[1] I. Stankov and A. Tsvetanov, "Matching text from start to finish against multiple regular expressions," in *32nd National Conference with International Participation "Telecom 2024"*, (Sofia, Bulgaria), pp. 21–22, IEEE, November 2024. 979-8-3503-5500-0/24.

[2] ISO/IEC, "Programming languages – c++," International Standard ISO/IEC 14882:2020, International Organization for Standardization, 2020.

[3] V. Falco, "Boost.beast: Http and websocket built on boost.asio." `https://www.boost.org/doc/libs/release/libs/beast/`, 2016. Accessed: 2024.

[4] T. An, "Drogon: A c++14/17/20 based http web application framework." `https://github.com/drogonframework/drogon`, 2018. Accessed: 2024.

[5] TechEmpower, "Web framework benchmarks." `https://www.techempower.com/benchmarks/`, 2024. Accessed: 2024.

[6] CrowCpp, "Crow: A fast and easy to use microframework for the web." `https://crowcpp.org/`, 2014. Accessed: 2024.

[7] S. Leonid, "Oat++: Modern web framework for c++." `https://oatpp.io/`, 2018. Accessed: 2024.

[8] Pistache Project, "Pistache: An elegant c++ rest framework." `http://pistache.io/`, 2015. Accessed: 2024.

[9] M. Belshe, R. Peon, and M. Thomson, "Hypertext transfer protocol version 2 (http/2)," RFC 7540, IETF, 2015. `https://tools.ietf.org/html/rfc7540`.

[10] I. Fette and A. Melnikov, "The websocket protocol," RFC 6455, IETF, 2011. `https://tools.ietf.org/html/rfc6455`.

[11] W. Glozer, "wrk - a http benchmarking tool." `https://github.com/wg/wrk`, 2012. Accessed: 2024.

[12] P. Nash and M. Horenovsky, "Catch2: A modern, c++-native, test framework for unit-tests." `https://github.com/catchorg/Catch2`, 2017. Accessed: 2024.

[13] jagt, "clumsy: An utility for simulating broken network for windows." `https://jagt.github.io/clumsy/`, 2014. Accessed: 2024. Uses WinDivert library for packet manipulation.